

บทที่ 3 : แบบจำลองระบบ

แนะนำแบบจำลองระบบ 01

แบบจำลองเชิงโครงสร้าง 02

แบบจำลองเชิงวัตถุ 03

1. แนะนำแบบจำลองระบบ

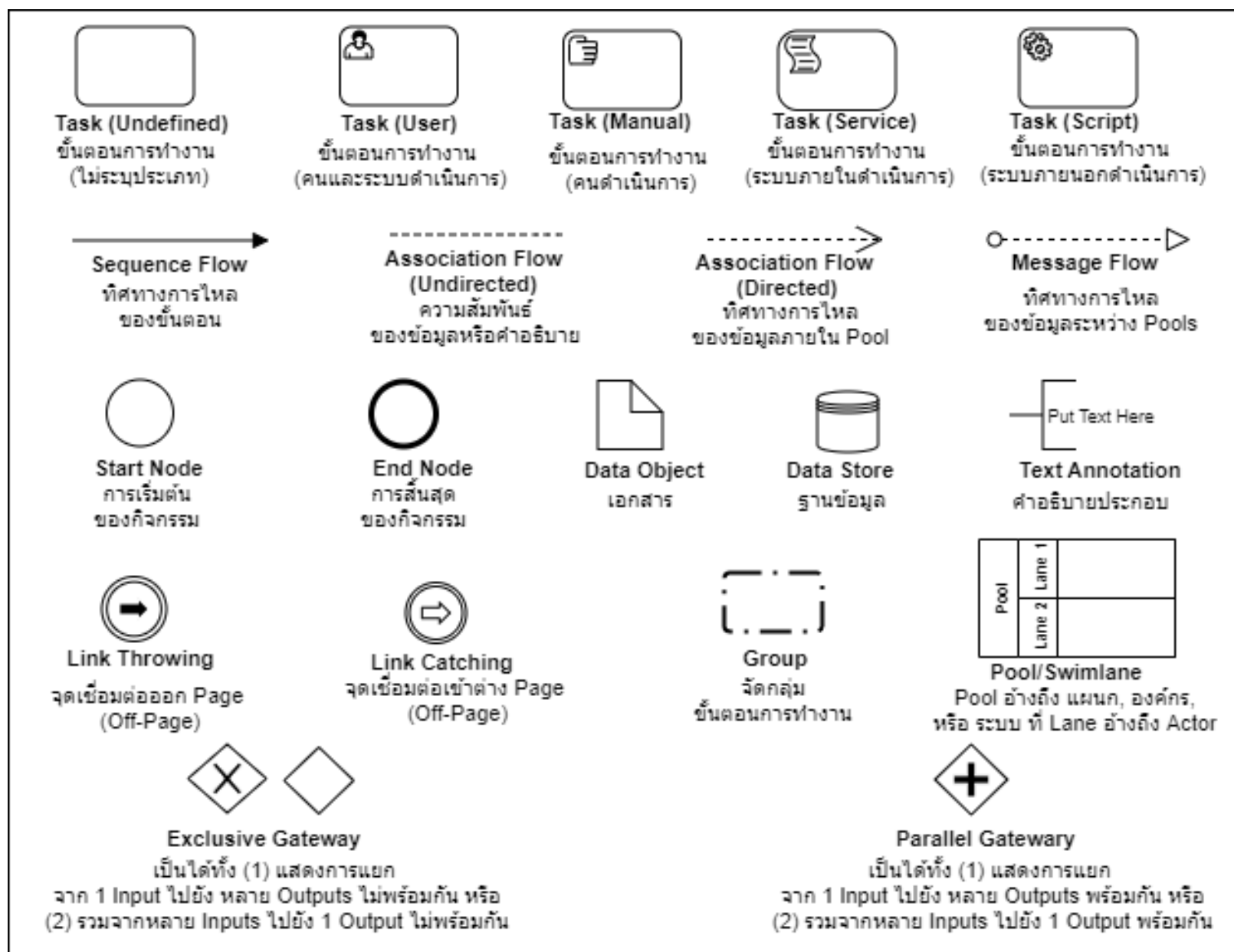
- แบบจำลองระบบ (System Model) เป็นสัญลักษณ์ที่ใช้จำลองข้อเท็จจริงต่าง ๆ ที่จะเกิดขึ้นในระบบ
- มีหลายชนิดเพื่อให้สอดคล้องกับมุมมองที่หลากหลายของผู้ใช้งาน
- เป็นเครื่องมือในการสื่อสารเพื่อให้เข้าใจตรงกัน
- นำไปใช้ทำ Project Plan (ส่วนของการประเมิน Size/Effort/Cost) ในขั้นการวางแผน (Planning)
- นำไปใช้ทำเอกสารข้อกำหนดความต้องการซอฟต์แวร์ (Software Requirement Specification : SRS) ในขั้นการวิเคราะห์ความต้องการ (Requirement Analysis)
- นำไปใช้ทำเอกสารออกแบบระบบ (Software Design Document : SDD) ในขั้นการออกแบบระบบ (Design)
- แบ่งออกเป็น แบบจำลองเชิงโครงสร้าง (Structured Model) และแบบจำลองเชิงวัตถุ (Object-Oriented Model)

2. แบบจำลองเชิงโครงสร้าง

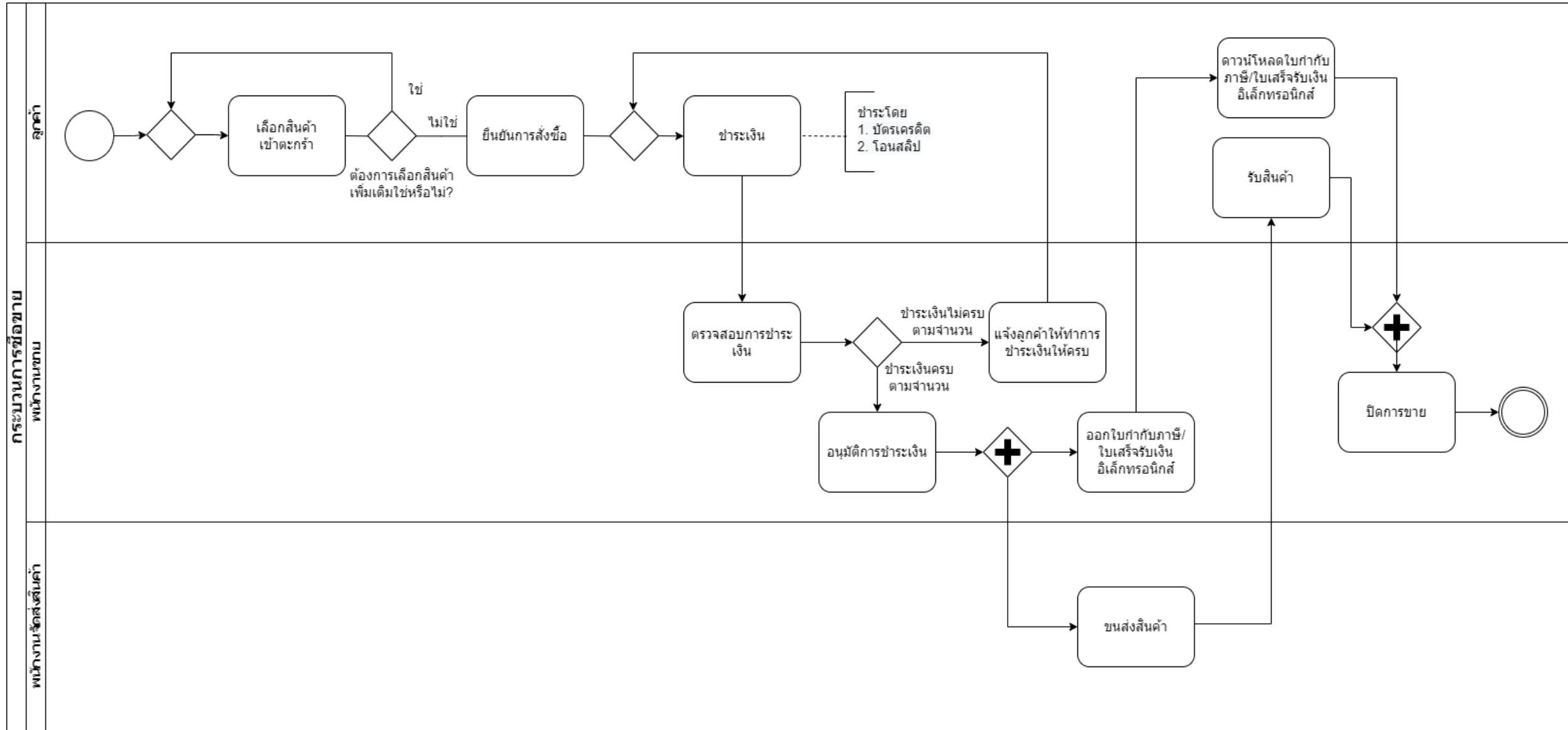
- แบบจำลองเชิงโครงสร้าง (Structured Model) เป็นแบบจำลองที่ใช้สำหรับการวิเคราะห์และออกแบบระบบเชิงโครงสร้าง (Structured System Analysis and Design)
- ใช้กับภาษาเชิงโครงสร้าง เช่น C, Pascal, Basic, และ COBOL
- เป็นแบบจำลองดั้งเดิม
- เน้น Process/Data
 - Process Model (Flowchart, BPMN, Context Diagram, DFD)
 - Data Model (ERD)

1.1 Business Process Modeling and Notation

- แบบจำลองและสัญลักษณ์กระบวนการธุรกิจ (Business Process Modeling and Notation: BPMN) คือมาตรฐานแบบจำลองกระบวนการปฏิบัติงาน
- พัฒนาโดย Business Process Management Initiative (BPMI) และ Object Management Group (OMG) ตั้งแต่ปี 2005
- มีรูปแบบคล้ายกับ Flowchart
- มีการปรับปรุงอย่างต่อเนื่องมากกว่า Flowchart



ภาพที่ 2-1 : สัญลักษณ์ของ BPMN



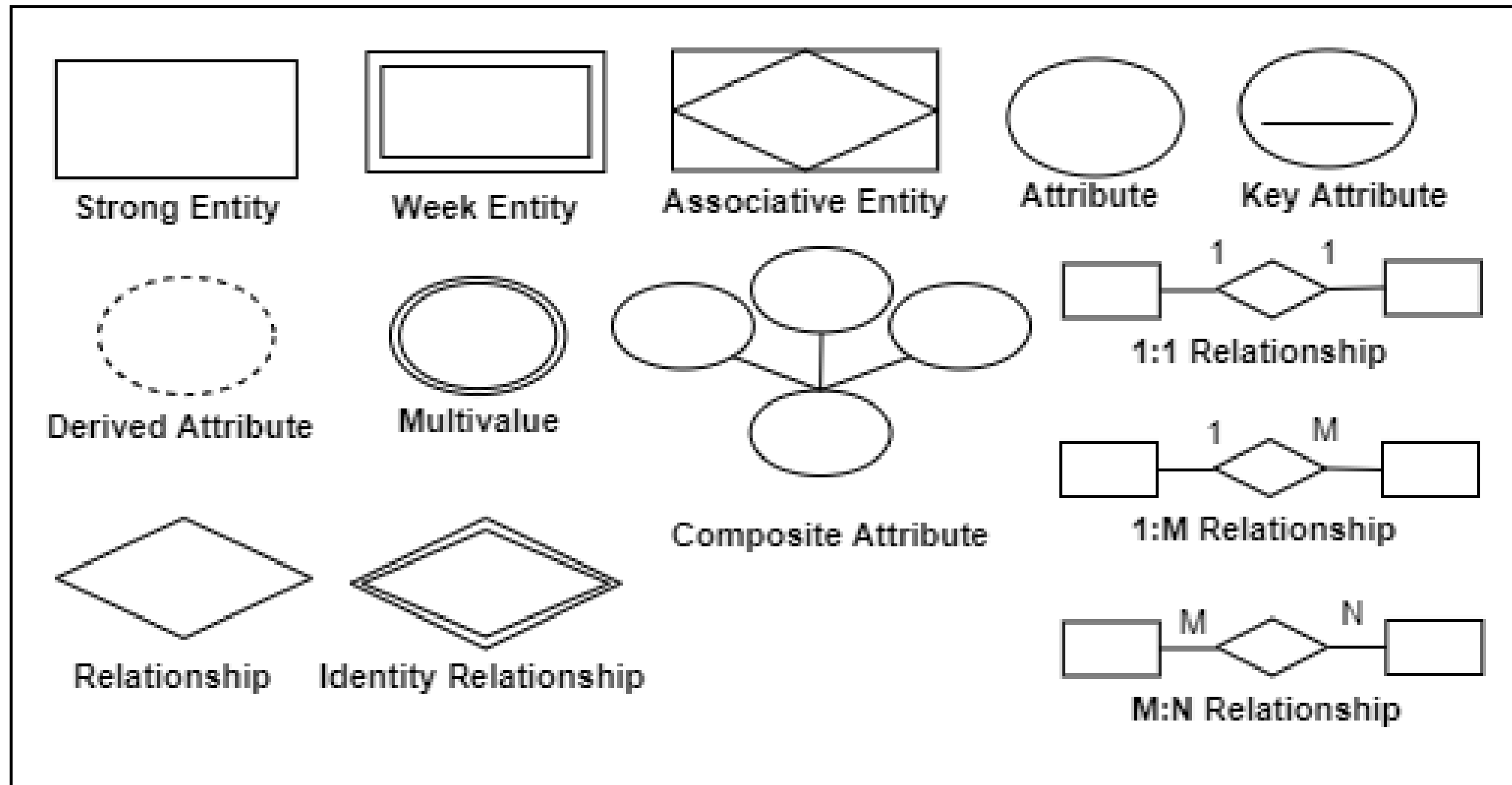
ภาพที่ 2-x : BPMN ของกระบวนการซื้อขาย

1.2 Data Flow Diagram

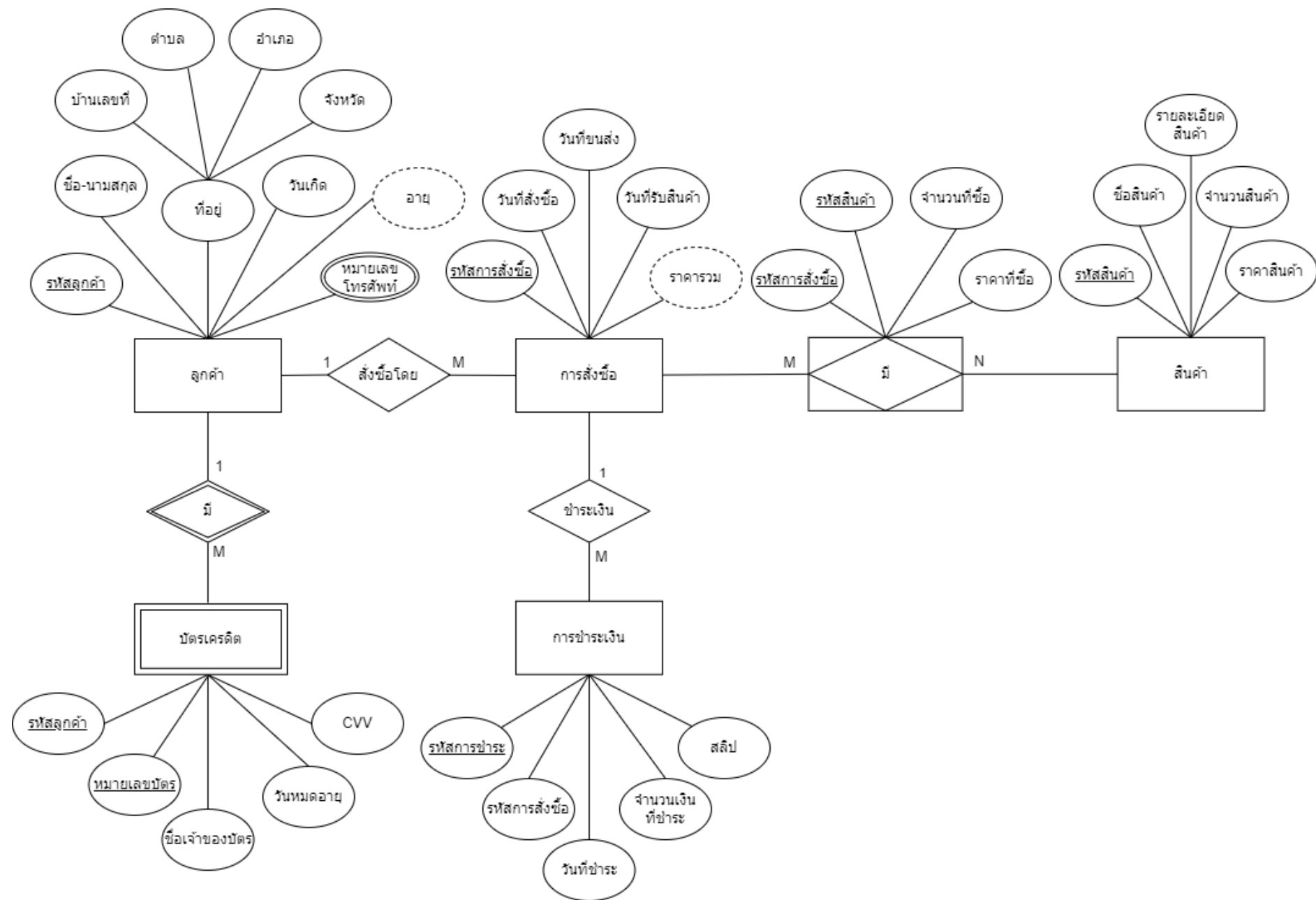
- **แผนภาพกระแสข้อมูล (Data Flow Diagram: DFD)**
เป็นแผนภาพแสดงทิศทางการไหลของข้อมูลในระบบจากกระบวนการหนึ่งไปยังกระบวนการหนึ่ง จากกระบวนการไปยังแหล่งจัดเก็บข้อมูล หรือจากกระบวนการไปยังผู้ที่เกี่ยวข้อง
- ในขั้นตอนการวางแผนและการวิเคราะห์ (**Requirement Analysis**) อาจใช้เพียง Context Diagram และ DFD ระดับ 0 โดยที่ระดับที่สูงกว่ามักเพิ่มเข้าไปในขั้นตอนการออกแบบ (**Design**)
- สามารถนำไปใช้สำหรับการคำนวณ Function Point เพื่อคำนวณหาขนาดซอฟต์แวร์ (Software Size) ค่าความพยายาม (Effort) และต้นทุน (Cost) ในขั้นตอนการวางแผนโครงการ (**Planning**)

1.3 Entity Relationship Diagram

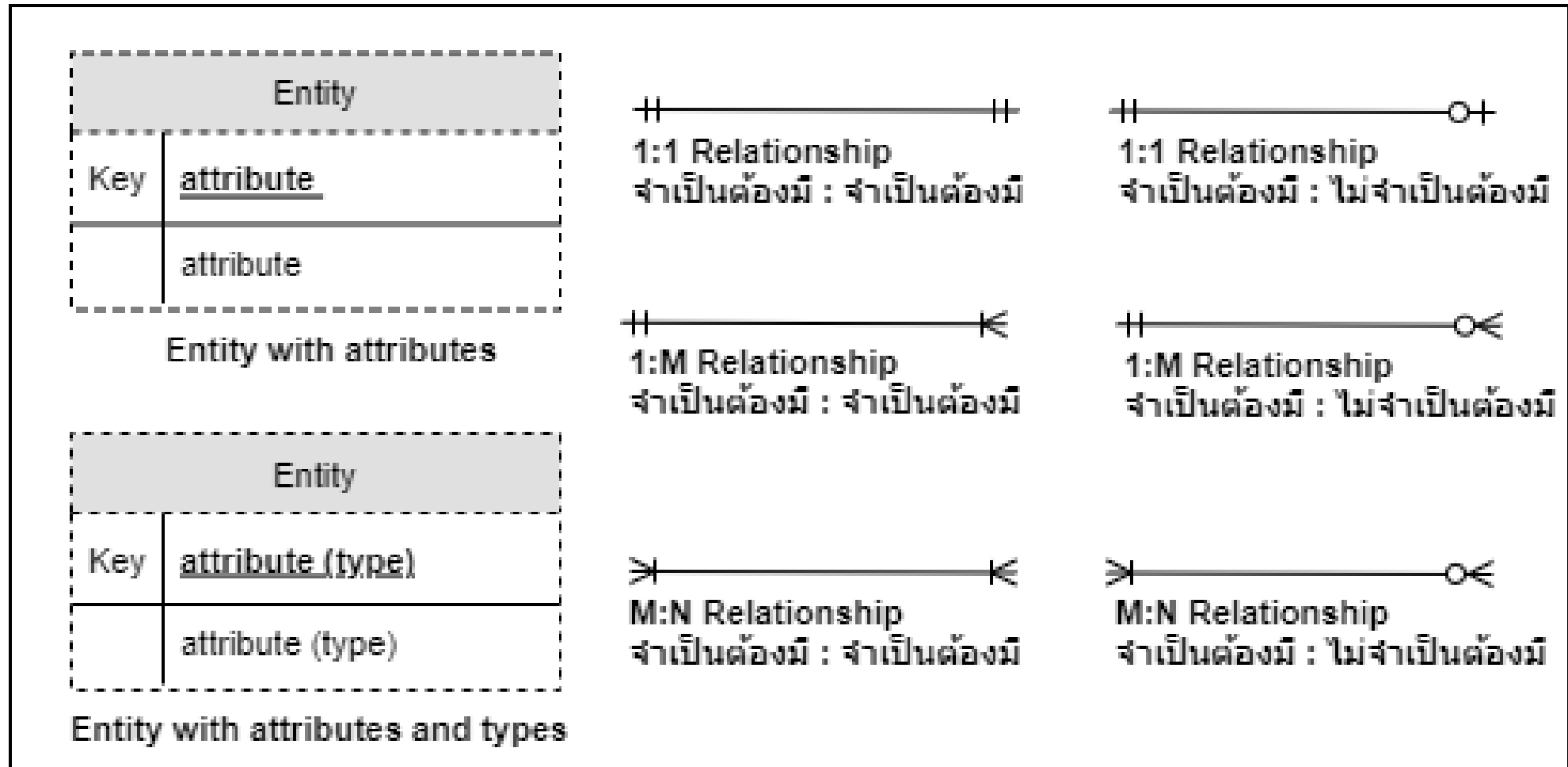
- แผนภาพความสัมพันธ์เอนทิตี (Entity Relationship Diagram : ERD)
เป็นแผนภาพแสดงความสัมพันธ์ระหว่าง Entity หรือตารางข้อมูล
- แต่ละ Entity จะประกอบไปด้วย Attribute หรือฟิลด์ข้อมูล
- ในขั้นตอนการวางแผนและการวิเคราะห์อาจใช้เพียง ERD ระดับ Logical โดยที่ระดับที่สูงกว่ามัก
เพิ่มเข้าไปในขั้นตอนการออกแบบ
- สามารถนำไปคำนวณ Function Point เพื่อหาขนาดซอฟต์แวร์ ค่าความพยายาม และต้นทุนใน
ขั้นตอนการวางแผนโครงการ



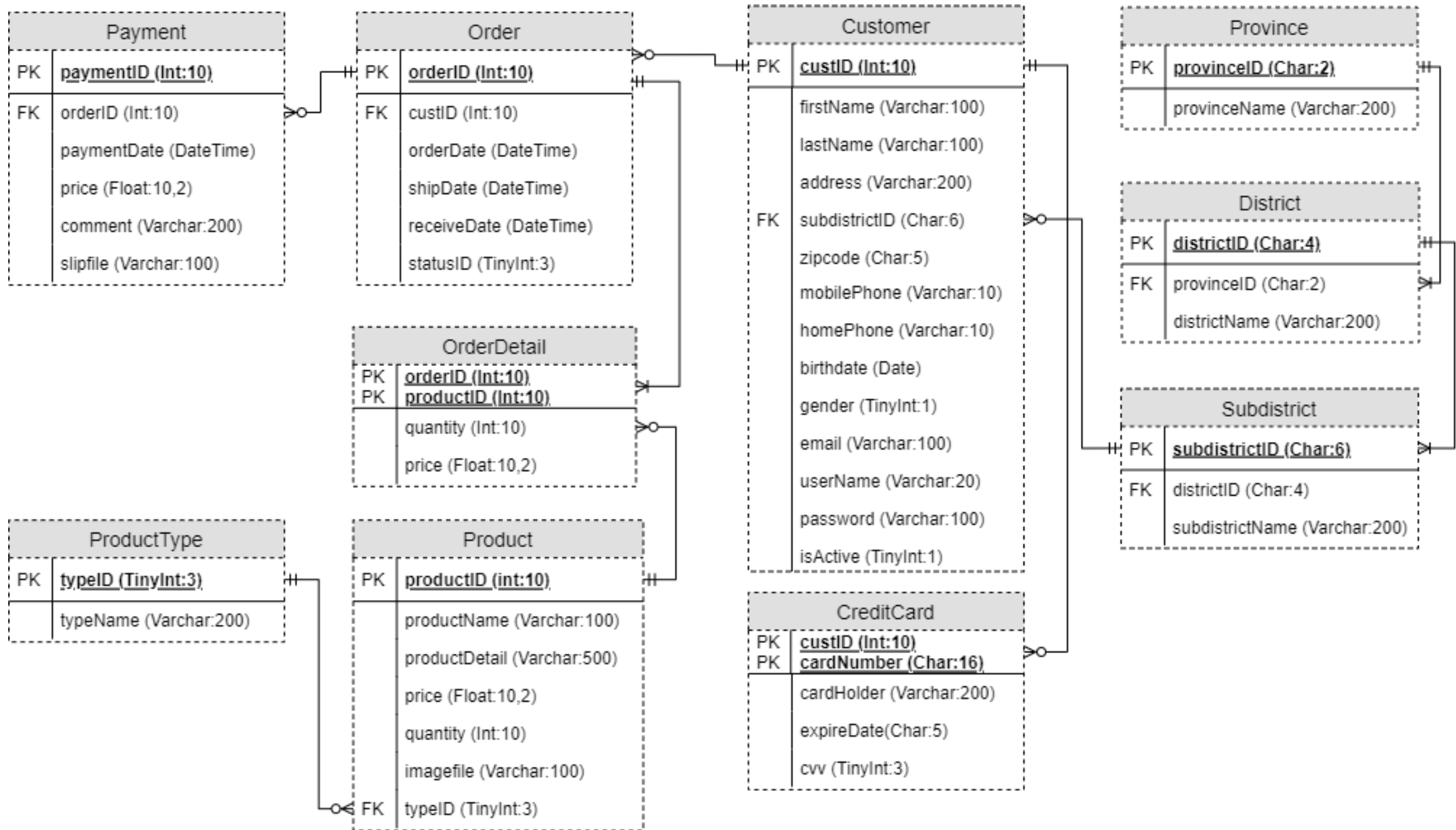
ภาพที่ 2-6 : สัญลักษณ์ ERD ในแบบของ Chen



ภาพที่ 2-6 : ERD ของระบบซื้อขายออนไลน์ ในแบบของ Chen



ภาพที่ 2-6 : สัญลักษณ์ ERD ในแบบของ Crow's foot



ภาพที่ 2-6 : ERD ของระบบซื้อขายออนไลน์ ในแบบของ Crow's foot

3. แบบจำลองเชิงวัตถุ

- แบบจำลองเชิงวัตถุ (Object-Oriented Model) ใช้สำหรับการวิเคราะห์และออกแบบระบบเชิงวัตถุ (Object Oriented System Analysis and Design)
- เน้น Object
- ใช้กับการพัฒนาด้วยภาษา Object-Oriented Programming (OOP) เช่น C++, C#, Java ฯลฯ

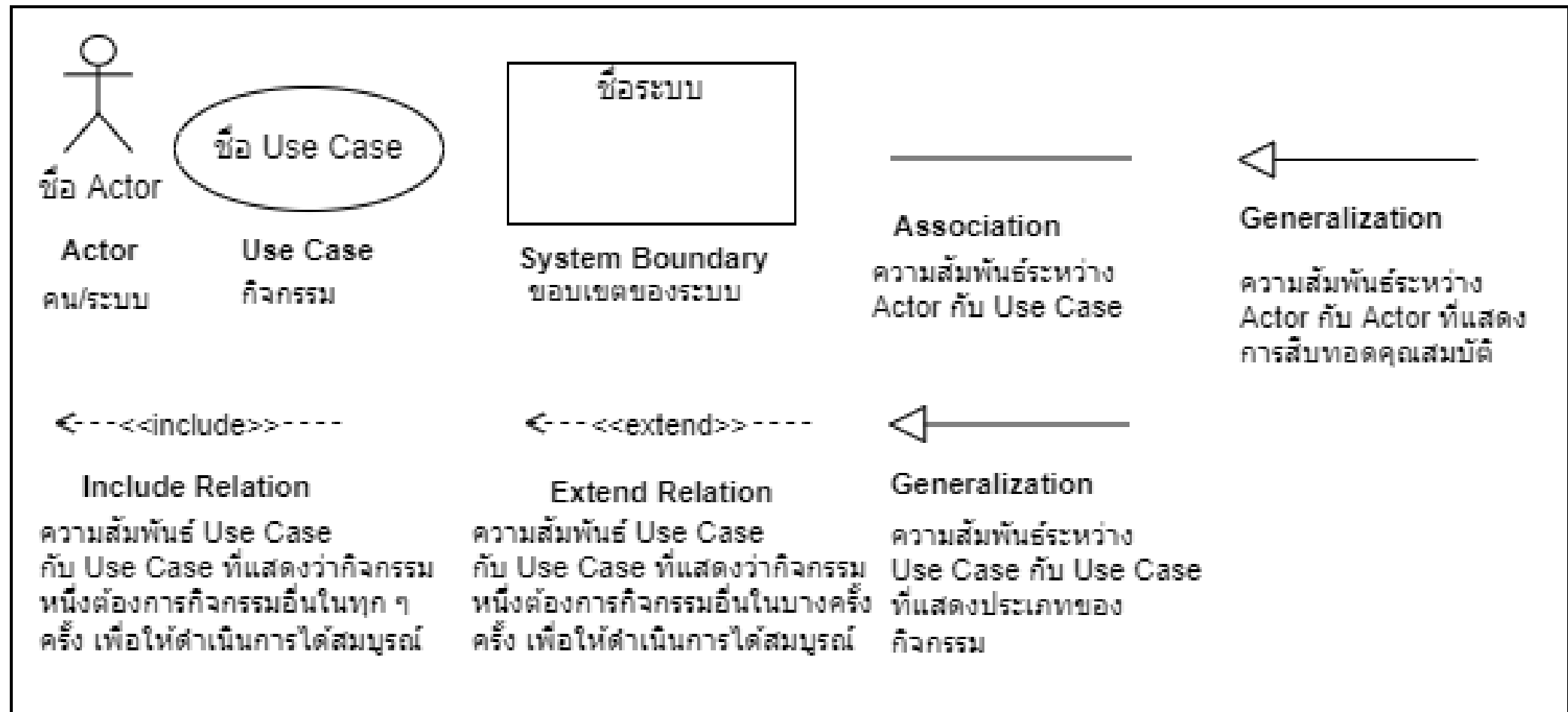
3.1 แนะนำยูเอ็มแอล

ยูเอ็มแอล (Unified Modeling Language : UML) เป็นภาษามาตรฐานในการสร้างแบบจำลอง ดูแลโดย Object Management Group (OMG) ซึ่งแบ่ง UML ออกเป็น 2 กลุ่ม

- ไคอะแกรมแบบโครงสร้าง (Structural Diagram)
 - Class Diagram
 - Object Diagram
 - Component Diagram
 - Deployment Diagram
- ไคอะแกรมแบบพฤติกรรม (Behavioral Diagram)
 - Use Case Diagram
 - Activity Diagram
 - Sequence Diagram
 - Collaboration Diagram
 - State Diagram

3.2 Use Case Diagram

- **ยูสเคสไดอะแกรม (Use Case Diagram)** เป็นไดอะแกรมสำหรับแสดงความสัมพันธ์ระหว่าง Actor กับ Use Case
- สามารถนำไปใช้ในการคำนวณ Use Case Point เพื่อหาขนาดซอฟต์แวร์ ค่าความพยายาม และต้นทุนในขั้นตอนการวางแผนโครงการ
- องค์ประกอบของ Use Case Diagram
 - Actor แทนคนหรือระบบอื่น
 - Use Case แทนกระบวนการ/กิจกรรม/ฟังก์ชันการทำงานของระบบ
 - System Boundary แทนขอบเขตของระบบ
 - Relationship แทนความสัมพันธ์

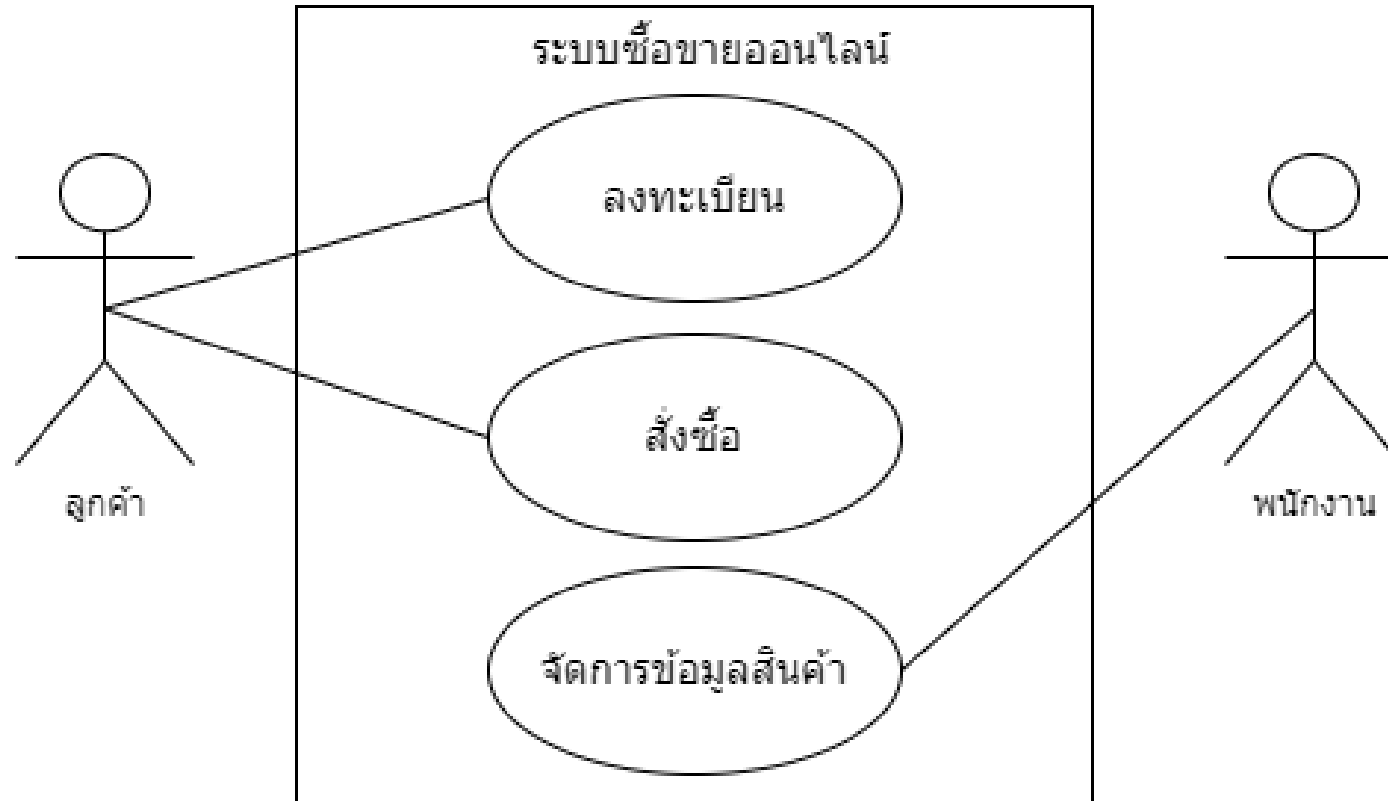


ภาพที่ 3-1 : สัญลักษณ์ของ Use Case Diagram

ความสัมพันธ์ (Relationship) แบ่งออกเป็น 4 ประเภท ดังนี้

- ความเกี่ยวข้องกัน (Association)
- การสืบทอดคุณสมบัติ (Generalization/Inheritance)
- การรวมเข้า (Include Relation)
- การขยาย (Extend Relation)

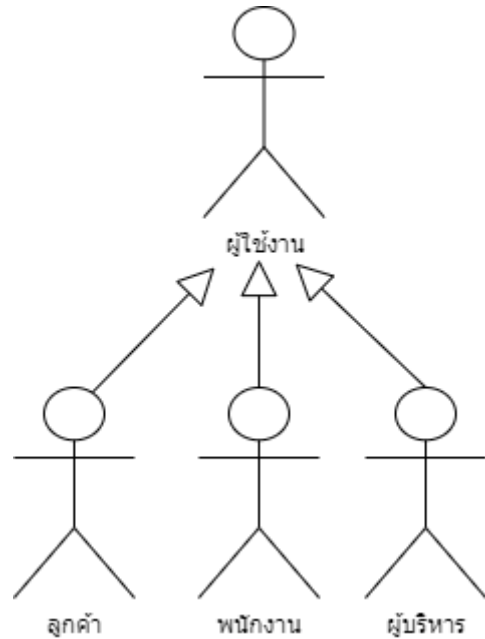
❑ ความเกี่ยวข้องกัน (Association) แทนความสัมพันธ์ระหว่าง Actor กับ Use Case



ภาพที่ 3-2 : ความสัมพันธ์แบบ Association ระหว่าง Actor กับ Use Case

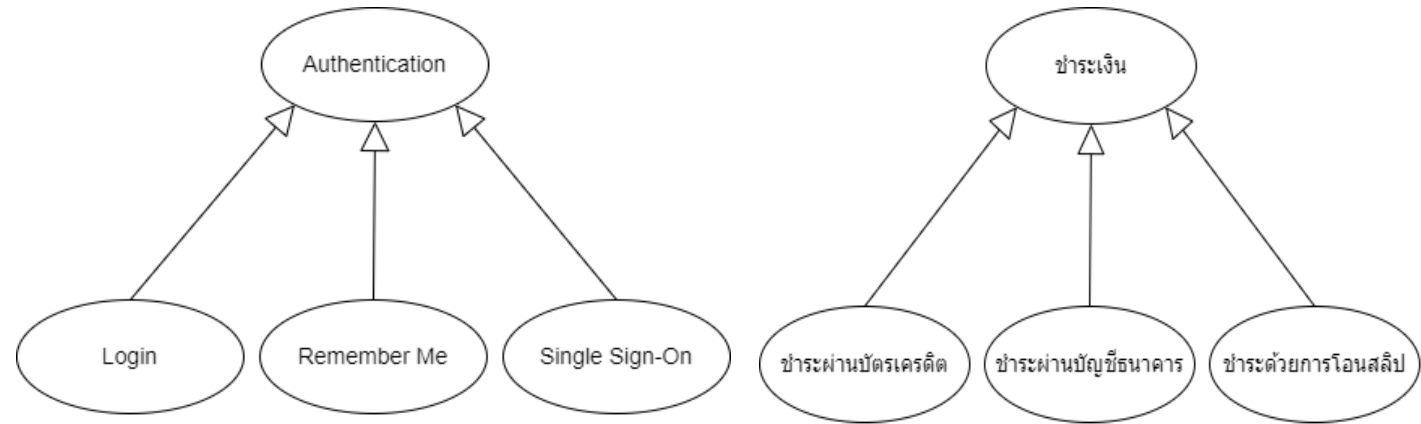
❑ การสืบทอดคุณสมบัติ (Generalization/Inheritance) แบ่งออกเป็น

- ความสัมพันธ์ที่แสดงสืบทอดคุณสมบัติ : ซึ่ง Actor ที่สืบทอดจะมีคุณสมบัติมากกว่า Actor หลัก
- ความสัมพันธ์ที่แสดงรายการของประเภท : ซึ่ง Use Case หนึ่งประกอบไปด้วยอีกหลาย Use Case



ภาพที่ 3-3 ก : ความสัมพันธ์แบบ

Generalization ระหว่าง Actor กับ Actor

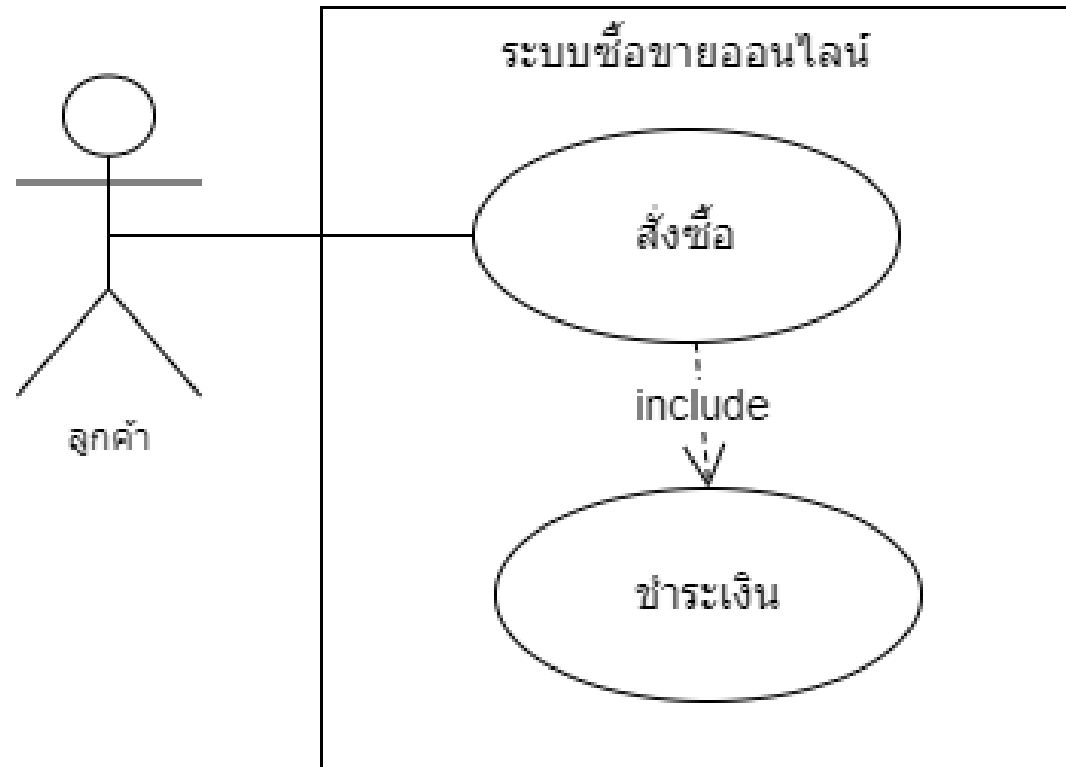


ภาพที่ 3-3 ข : ความสัมพันธ์แบบ

Generalization ระหว่าง Use Case กับ Use Case

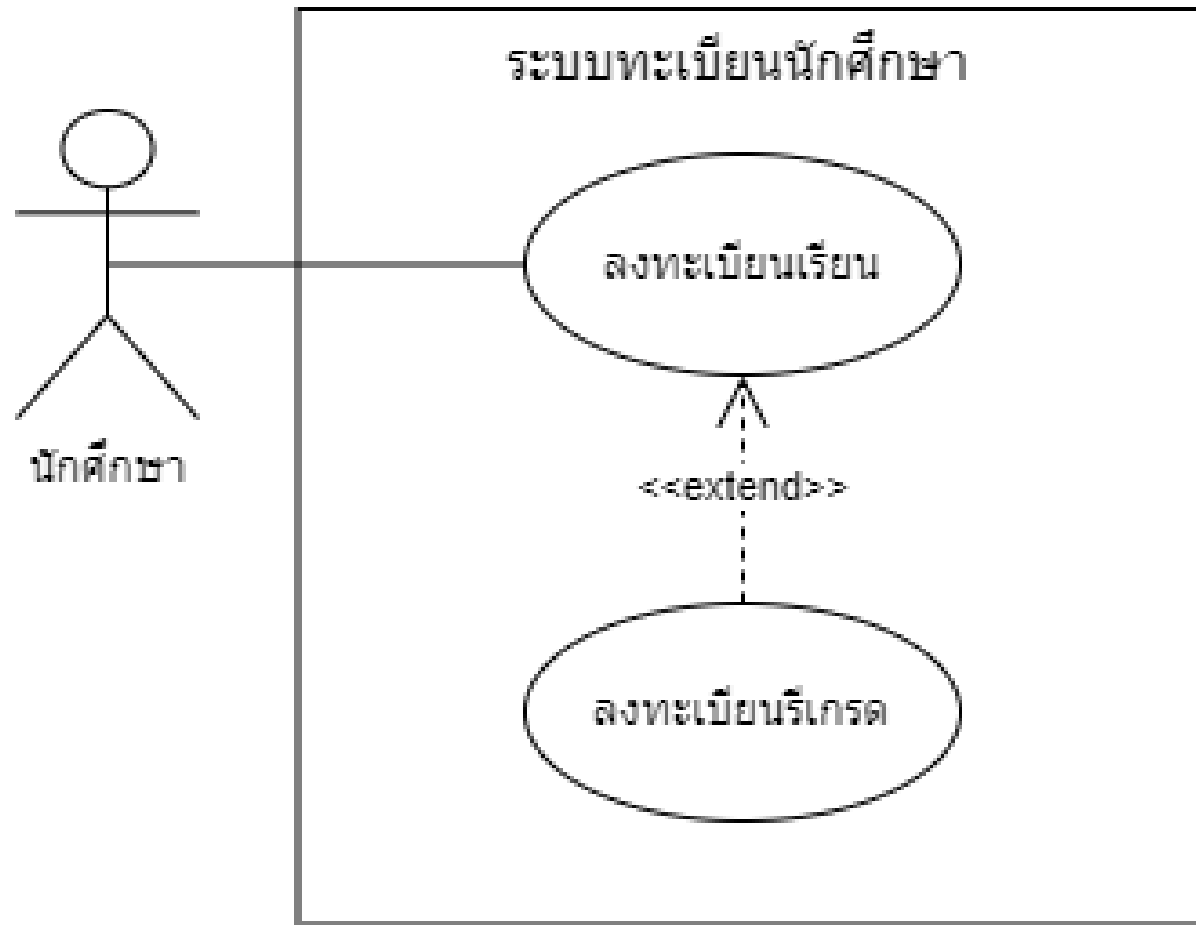
ภาพที่ 3-3 : ความสัมพันธ์แบบ Generalization

❑ การรวมเข้า (Include Relation) แทนความสัมพันธ์ Use Case กับ Use Case ที่แสดงว่ากระบวนการหนึ่งต้องการกระบวนการอื่นในทุก ๆ ครั้ง เพื่อให้ดำเนินการได้สมบูรณ์

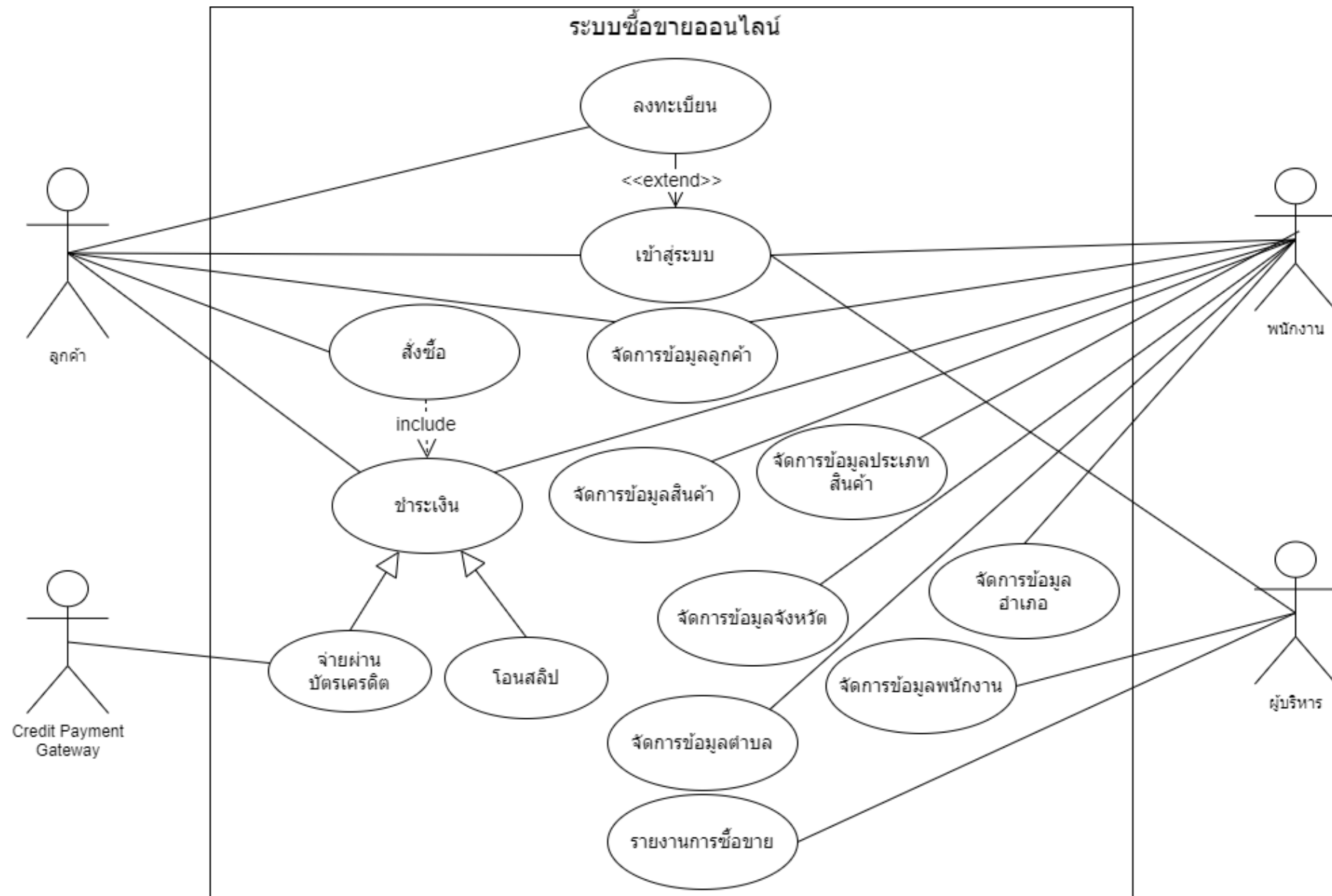


ภาพที่ 3-4 : ความสัมพันธ์แบบ Include Relation

❑ การขยาย (Extend Relation) แทนความสัมพันธ์ระหว่าง Use Case กับ Use Case ที่แสดงว่ากระบวนการหนึ่งต้องการกระบวนการอื่นในบางครั้ง เพื่อให้ดำเนินการได้สมบูรณ์



ภาพที่ 3-5 : ความสัมพันธ์แบบ Extend Relation



ภาพที่ 3-6 : Use Case ของระบบซื้อขายออนไลน์

3.3 Use Case Diagram Description

คำอธิบายยูสเคส (Use Case Description) เป็นการอธิบายรายละเอียดของ Use Case มีองค์ประกอบดังนี้

- Use Case Name
- Description
- Pre-Condition
- Post-Condition
- Flows of Events
- Basic Flows of Events
- Alternative Flows of Events

Use Case Name:	ลงทะเบียน	
Description:	ลงทะเบียนสำหรับเข้าใช้งานระบบ	
Actors:	ลูกค้า	
Pre-Conditions:	ลูกค้ายังไม่เป็นสมาชิก	
Post-Conditions:	ลูกค้าลงทะเบียนสำเร็จ	
Basic Flows of Events:	Actor	System
	1. คลิกปุ่ม "สมัครสมาชิก" จากหน้าหลักหรือหน้าเข้าสู่ระบบ 3. ระบบผู้ใช้ รหัสผ่าน ชื่อแรก นามสกุล ฯลฯ แล้วคลิกปุ่ม "สมัครสมาชิก"	2. แสดงหน้าจอสมัครสมาชิก 4. ตรวจสอบข้อมูลที่จำเป็นและรูปแบบข้อมูล 5. ตรวจสอบชื่อผู้ใช้งาน 6. บันทึกข้อมูลสมาชิกลงฐานข้อมูล 7. แจ้งการลงทะเบียนสำเร็จแล้ว
Alternative Flows of Events:		4.1 แจ้งรายการข้อมูลที่ไม่ได้ระบุหรือรูปแบบข้อมูลผิดพลาด เช่น นามสกุลไม่ได้ระบุ, รูปแบบอีเมลผิด, รูปแบบวันที่ผิด ฯลฯ 5.1 แจ้งชื่อผู้ใช้งาน

ภาพที่ 3-7 : Use Case Description ของการลงทะเบียน

Use Case Name:	เข้าสู่ระบบ	
Description:	เข้าสู่ระบบเพื่อให้สามารถซื้อสินค้าได้	
Actors:	ลูกค้า/พนักงาน/ผู้บริหาร	
Pre-Conditions:	ลูกค้ายังไม่ทำการเข้าสู่ระบบ	
Post-Conditions:	เข้าสู่ระบบสำเร็จ	
Basic Flows of Events:	Actor	System
	1. คลิกปุ่ม "เข้าสู่ระบบ" จากหน้าหลัก 3. ระบุชื่อผู้ใช้และรหัสผ่าน แล้วคลิกปุ่ม "เข้าสู่ระบบ"	2. แสดงหน้าจอเข้าสู่ระบบเพื่อให้ระบุชื่อผู้ใช้และรหัสผ่าน 4. ตรวจสอบความครบถ้วนของชื่อผู้ใช้และรหัสผ่าน 5. ตรวจสอบสิทธิ์การใช้งาน 6. รีไตเรกไปยังหน้าแรกของระบบ
Alternative Flows of Events:		4.1 แจ้งรายการข้อมูลที่ไม่ได้ระบุ 5.1 แจ้งชื่อผู้ใช้และรหัสผ่านไม่ถูกต้อง

ภาพที่ 3-8 : Use Case Description ของการเข้าสู่ระบบ

Use Case Name:	จัดการข้อมูลลูกค้า	
Description:	แก้ไขและลบข้อมูลลูกค้า	
Actors:	พนักงาน	
Pre-Conditions:	- เข้าสู่ระบบเรียบร้อยแล้ว - เข้าสู่หน้าแสดงรายการลูกค้า	
Post-Conditions:	แก้ไขหรือลบข้อมูลสำเร็จ	
Basic Flows of Events:	Actor	System
	1. แก้ไขข้อมูลลูกค้า	
	1.1 เลือกลูกค้าที่ต้องการแก้ไข 1.3 แก้ไขรายละเอียดลูกค้า	1.2 แสดงรายละเอียดลูกค้า 1.4 ตรวจสอบข้อมูลที่จำเป็นและรูปแบบข้อมูล 1.5 บันทึกข้อมูลลูกค้าลงฐานข้อมูล 1.6 แจ้งการแก้ไขข้อมูลสำเร็จ
Alternative Flows of Events:	2. ลบข้อมูลลูกค้า	
	2.1 เลือกลูกค้าที่ต้องการลบ 2.3 คลิกยืนยันการลบ	2.2 แจ้งยืนยันการลบข้อมูล 2.4 ลบข้อมูลออกจากฐานข้อมูล 2.5 แจ้งการลบข้อมูลสำเร็จ
	4. แจ้งเตือนความผิดพลาด	
		1.4.1 แจ้งรายการข้อมูลที่ไม่ได้ระบุหรือรูปแบบข้อมูลผิดพลาด

ภาพที่ 3-9 : Use Case Description ของการจัดการข้อมูลลูกค้า

Use Case Name:	จัดการข้อมูลพนักงาน	
Description:	เพิ่ม แก้ไข และลบข้อมูลพนักงาน	
Actors:	ผู้บริหาร	
Pre-Conditions:	- เข้าสู่ระบบเรียบร้อยแล้ว - เข้าสู่หน้าแสดงรายการพนักงาน	
Post-Conditions:	เพิ่ม แก้ไข ลบข้อมูลสำเร็จ	
Basic Flows of Events:	Actor	System
	1. เพิ่มข้อมูลพนักงาน 1.1 คลิกปุ่มเพิ่มข้อมูลพนักงาน 1.3 ระบุรายละเอียดพนักงาน	1.2 แสดงหน้าจอการเพิ่มข้อมูลพนักงาน 1.4 ตรวจสอบข้อมูลที่จำเป็นและรูปแบบข้อมูล 1.5 ตรวจสอบการซ้ำกันของข้อมูลพนักงาน 1.6 บันทึกข้อมูลพนักงานลงฐานข้อมูล 1.7 แจ้งการเพิ่มข้อมูลสำเร็จ
Alternative Flows of Events:	2. แก้ไขข้อมูลพนักงาน	
	2.1 เลือกพนักงานที่ต้องการแก้ไข 2.3 แก้ไขรายละเอียดพนักงาน	2.2 แสดงรายละเอียดพนักงาน 2.4 ตรวจสอบข้อมูลที่จำเป็นและรูปแบบข้อมูล 2.5 บันทึกข้อมูลพนักงานลงฐานข้อมูล 2.6 แจ้งการแก้ไขข้อมูลสำเร็จ
	3. ลบข้อมูลพนักงาน	
	3.1 เลือกพนักงานที่ต้องการลบ 3.3 ทำการยืนยันการลบ	3.2 แจ้งเตือนให้ทำการยืนยันการลบ 3.4 ลบข้อมูลออกจากฐานข้อมูล 3.5 แจ้งการลบข้อมูลสำเร็จ
	4. แจ้งเตือนความผิดพลาด	
		1.4.1 แจ้งข้อมูลที่ไม่ได้ระบุหรือรูปแบบข้อมูลผิดพลาด 1.5.1 แจ้งข้อมูลพนักงานซ้ำ 2.4.1 แจ้งข้อมูลที่ไม่ได้ระบุหรือรูปแบบข้อมูลผิดพลาด

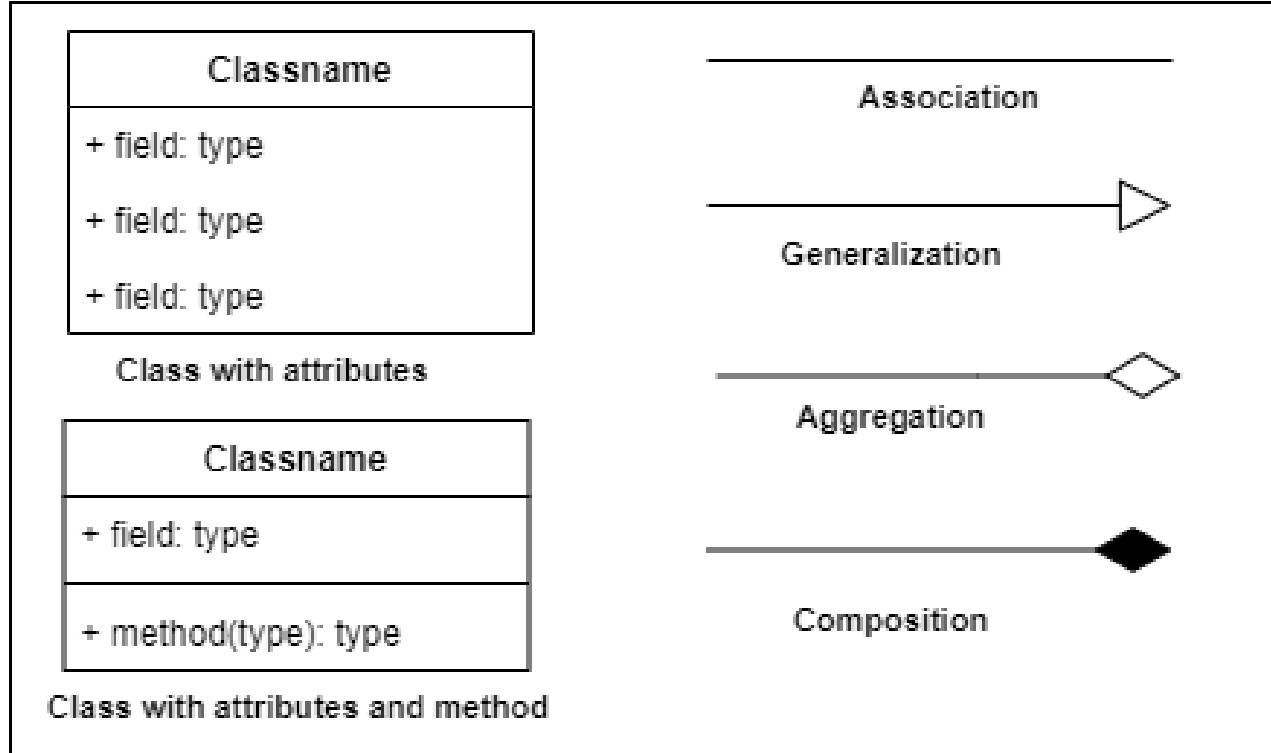
ภาพที่ 3-10 : Use Case Description ของการจัดการข้อมูลพนักงาน

Use Case Name:	จัดการข้อมูลสินค้า	
Description:	เพิ่ม แก้ไข และลบข้อมูลสินค้า	
Actors:	ผู้บริหาร	
Pre-Conditions:	- เข้าสู่ระบบเรียบร้อยแล้ว - เข้าสู่หน้าแสดงรายการสินค้า	
Post-Conditions:	เพิ่ม แก้ไข ลบข้อมูลสำเร็จ	
Basic Flows of Events:	Actor	System
	1. เพิ่มข้อมูลสินค้า 1.1 คลิกปุ่มเพิ่มข้อมูลสินค้า 1.3 ระบุรายละเอียดสินค้า	1.2 แสดงหน้าจอการเพิ่มข้อมูลสินค้า 1.4 ตรวจสอบข้อมูลที่จำเป็นและรูปแบบข้อมูล 1.5 ตรวจสอบการซ้ำกันของข้อมูลสินค้า 1.6 บันทึกข้อมูลสินค้าลงฐานข้อมูล 1.7 แจ้งการเพิ่มข้อมูลสำเร็จ
Alternative Flows of Events:	2. แก้ไขข้อมูลสินค้า	
	2.1 เลือกสินค้าที่ต้องการแก้ไข 2.3 แก้ไขรายละเอียดสินค้า	2.2 แสดงรายละเอียดสินค้า 2.4 ตรวจสอบข้อมูลที่จำเป็นและรูปแบบข้อมูล 2.5 บันทึกข้อมูลสินค้าลงฐานข้อมูล 2.6 แจ้งการแก้ไขข้อมูลสำเร็จ
	3. ลบข้อมูลสินค้า	
	3.1 เลือกสินค้าที่ต้องการลบ 3.3 ทำการยืนยันการลบ	3.2 แจ้งเตือนให้ทำการยืนยันการลบ 3.4 ลบข้อมูลออกจากฐานข้อมูล 3.5 แจ้งการลบข้อมูลสำเร็จ
	4. แจ้งเตือนความผิดพลาด	
		1.4.1 แจ้งข้อมูลที่ไม่ได้ระบุหรือรูปแบบข้อมูลผิดพลาด 1.5.1 แจ้งข้อมูลสินค้าซ้ำ 2.4.1 แจ้งข้อมูลที่ไม่ได้ระบุหรือรูปแบบข้อมูลผิดพลาด

ภาพที่ 3-11 : Use Case Description ของการจัดการข้อมูลสินค้า

3.4 Class Diagram

คลาสไดอะแกรม (Class Diagram) เป็นแผนภาพที่ใช้สำหรับแสดงความสัมพันธ์ระหว่าง Classes



ภาพที่ 3-12 : สัญลักษณ์ของ Class Diagram

❑ องค์ประกอบของ Class

- Class แสดงความเป็นวัตถุ (Object) ของระบบ เช่น คน สิ่งของ สถานที่
- Attribute แสดงคุณสมบัติของวัตถุ เช่น น้ำหนัก ส่วนสูง สีผิว
- Method แสดงกิจกรรมของวัตถุ เช่น วิ่ง นั่ง เดิน

Customer
+ CustID: Int
+ CustName: String
+ Address: Int
+ creditRating()

❑ การตั้งชื่อ Class

- ชื่อ Class ควรจะเป็นคำนาม (Noun) และขึ้นต้นด้วยตัวใหญ่ เช่น Customer, Employee, Product ฯลฯ
- ชื่อ Method ควรจะเป็นคำกริยา (Verb) และขึ้นต้นด้วยตัวเล็ก เช่น addCustomer, updateCustomer, deleteCustomer ฯลฯ

❑ การมองเห็น (Visibility) ของ Attribute และ Method

- **Public (+)** เป็นการอนุญาตให้ Class อื่นสามารถมองเห็น Attribute หรือ Method ภายใน Class ได้
- **Private (-)** เป็นการป้องกันไม่ให้ Class อื่นสามารถมองเห็น Attribute หรือ Method ภายใน Class ได้
- **Protect (#)** เป็นการอนุญาตให้เฉพาะ Class ที่ Inherit ไปสามารถมองเห็น Attribute หรือ Method ภายใน Class ได้

❑ ประเภทข้อมูล (Data Type) ของ Attribute

- **Integer** สำหรับ Attribute ที่เป็นจำนวนเต็ม
- **Real** สำหรับ Attribute ที่เป็นจำนวนจริง
- **String** สำหรับ Attribute ที่เป็นข้อความ
- **Boolean** สำหรับ Attribute ที่มีค่าความจริง เป็นจริงหรือเท็จ

Customer
+ CustID: Int
+ CustName: String
+ Address: Int
+ creditRating()

❑ การระบุจำนวนสมาชิกในความสัมพันธ์ระหว่าง Class (Multiplicity/Cardinality)

0..1 คือ มี 0 หรือ 1 แถว

1..1 คือ มี 1 แถว

1 คือ มี 1 แถว

0..* คือ มี 0 หรือ หลายแถว

***** คือ มี 0 หรือ หลายแถว

1..* คือ มี 1 หรือ หลายแถว

□ ความสัมพันธ์ (Relationship) แบ่งออกเป็น 4 ประเภท ดังนี้

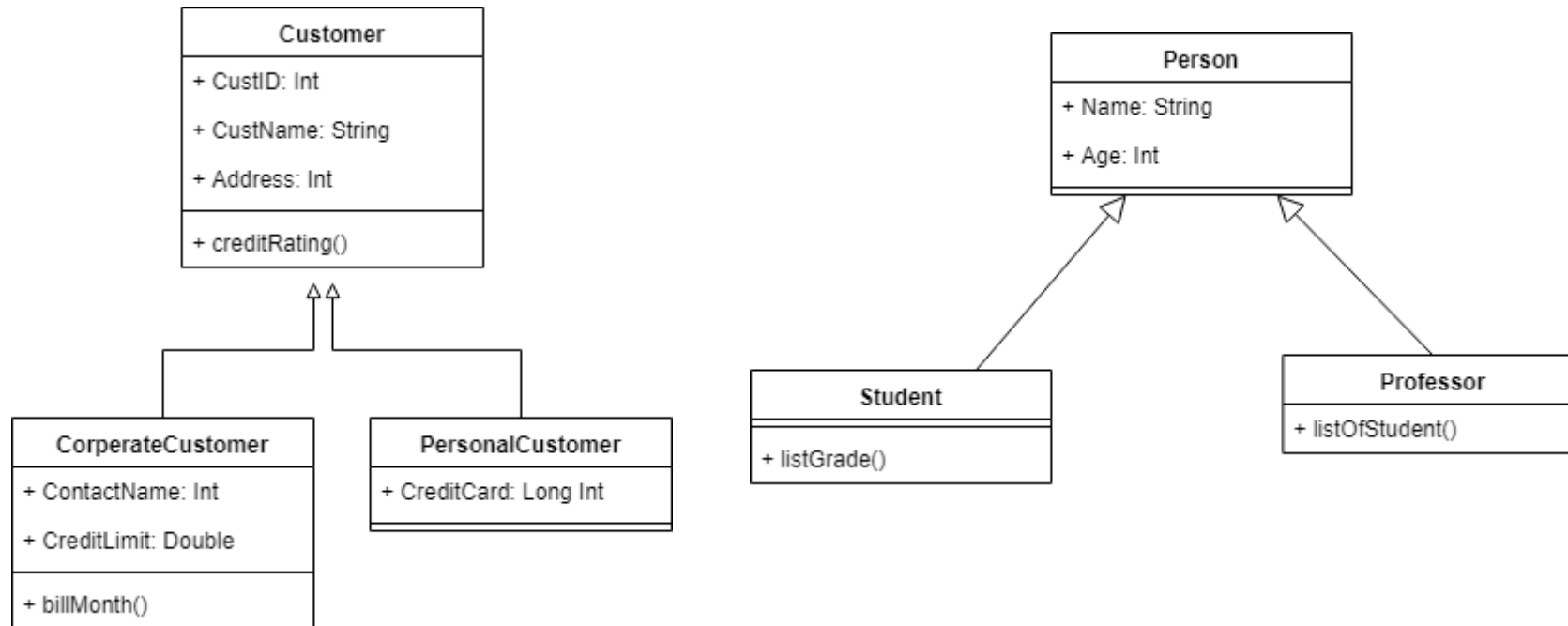
- ความเกี่ยวข้องกัน (Association)
- การสืบทอดคุณสมบัติ (Generalization/Inheritance)
- การเป็นองค์ประกอบของ (Composition)
- การเป็นส่วนหนึ่งของ (Aggregation)

- ❑ ความเกี่ยวข้องกัน (Association) เป็นความสัมพันธ์ระหว่าง Class ในระดับเดียวกัน ไม่มีการระบุการสืบทอดกัน การเป็นส่วนหนึ่งส่วนใดของกัน หรือมีความสำคัญที่มากกว่ากัน



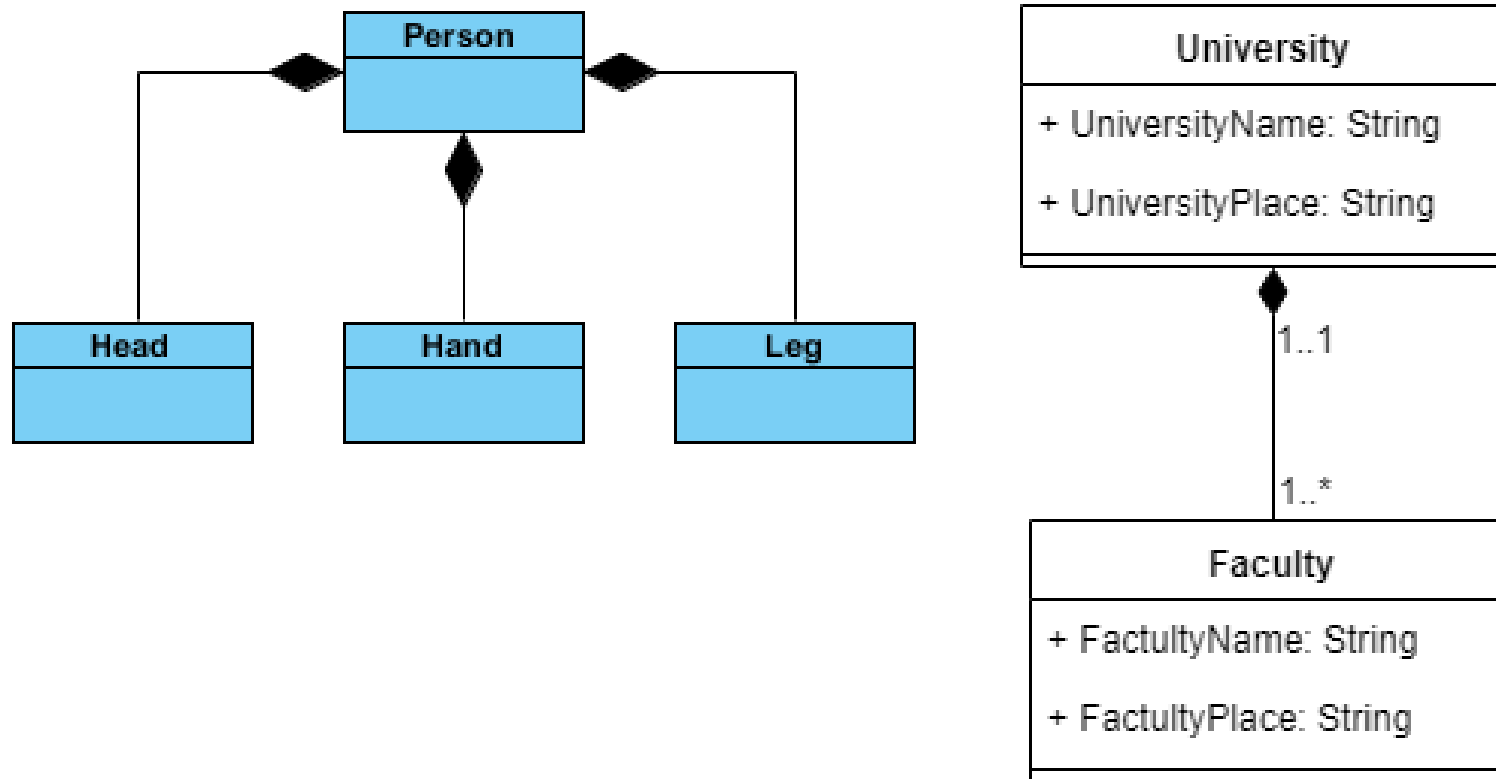
ภาพที่ 3-13 : ความสัมพันธ์แบบ Association

❑ การสืบทอดคุณสมบัติ (Generalization/Inheritance) เป็นความสัมพันธ์ระหว่าง Class แบบการสืบทอด (Inheritance) นั่นคือ Class ที่สืบทอด (Sub Class) มาต้องมีคุณสมบัติที่มากกว่า Class แม่ (Super Class) เช่น Class พนักงานประจำ Class พนักงาน Part-time และ Class ที่ปรึกษา สืบทอดจาก Class Employee



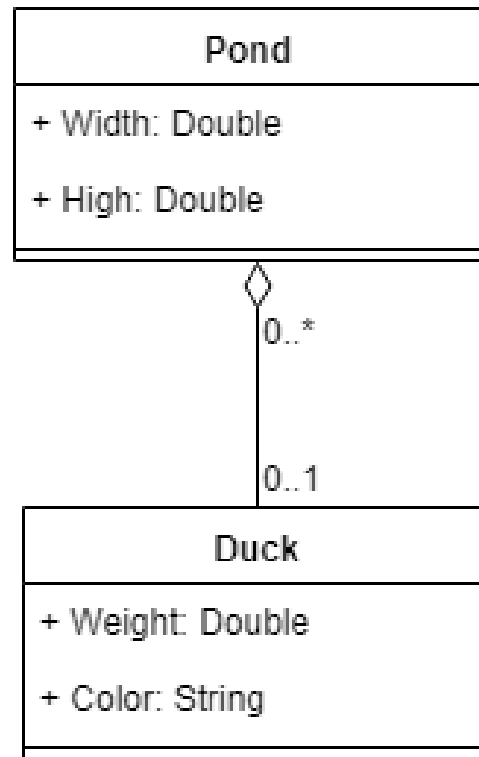
ภาพที่ 3-14 : ความสัมพันธ์แบบ Generalization

❑ การเป็นองค์ประกอบของ (Composition) เป็นความสัมพันธ์ระหว่าง Class ที่ว่า Class หนึ่งเป็นองค์ประกอบของอีก Class หนึ่ง เมื่อ Class หลักถูกทำลาย Class ที่เป็นองค์ประกอบจะถูกทำลายไปด้วย หรือเมื่อ Class ที่เป็นองค์ประกอบถูกทำลายก็จะทำให้ Class หลักสูญเสียคุณสมบัติไป

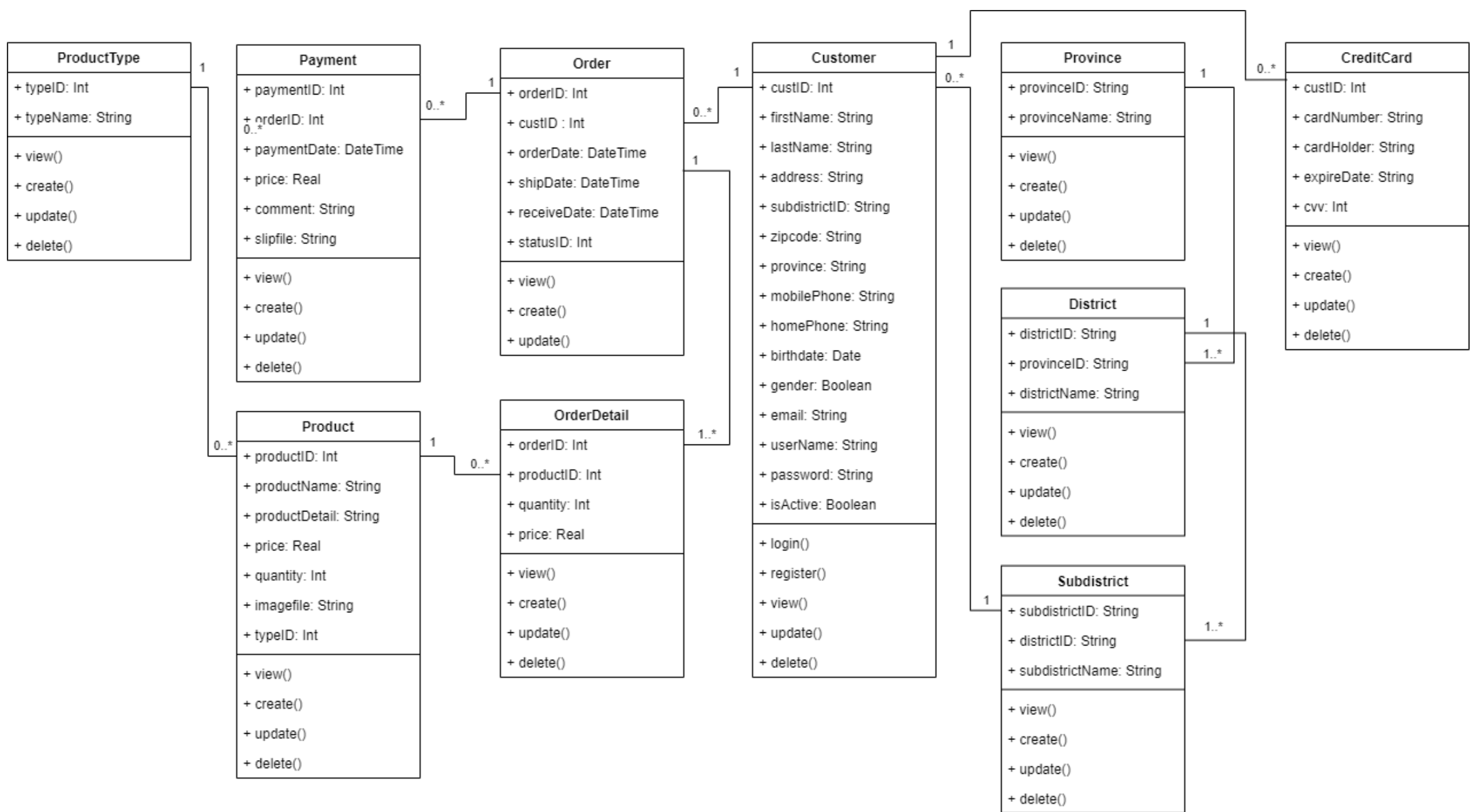


ภาพที่ 3-15 : ความสัมพันธ์แบบ Composition

❑ การเป็นส่วนหนึ่งของ (Aggregation) เป็นความสัมพันธ์ระหว่าง Class ที่ว่า Class หนึ่งเป็นส่วนหนึ่งของอีก Class หนึ่ง เมื่อ Class หลักถูกทำลาย Class ที่เป็นส่วนหนึ่งจะไม่ถูกทำลายไปด้วย หรือเมื่อ Class ที่เป็นส่วนหนึ่งถูกทำลายก็จะไม่ทำให้ Class หลักสูญเสียคุณสมบัติไป



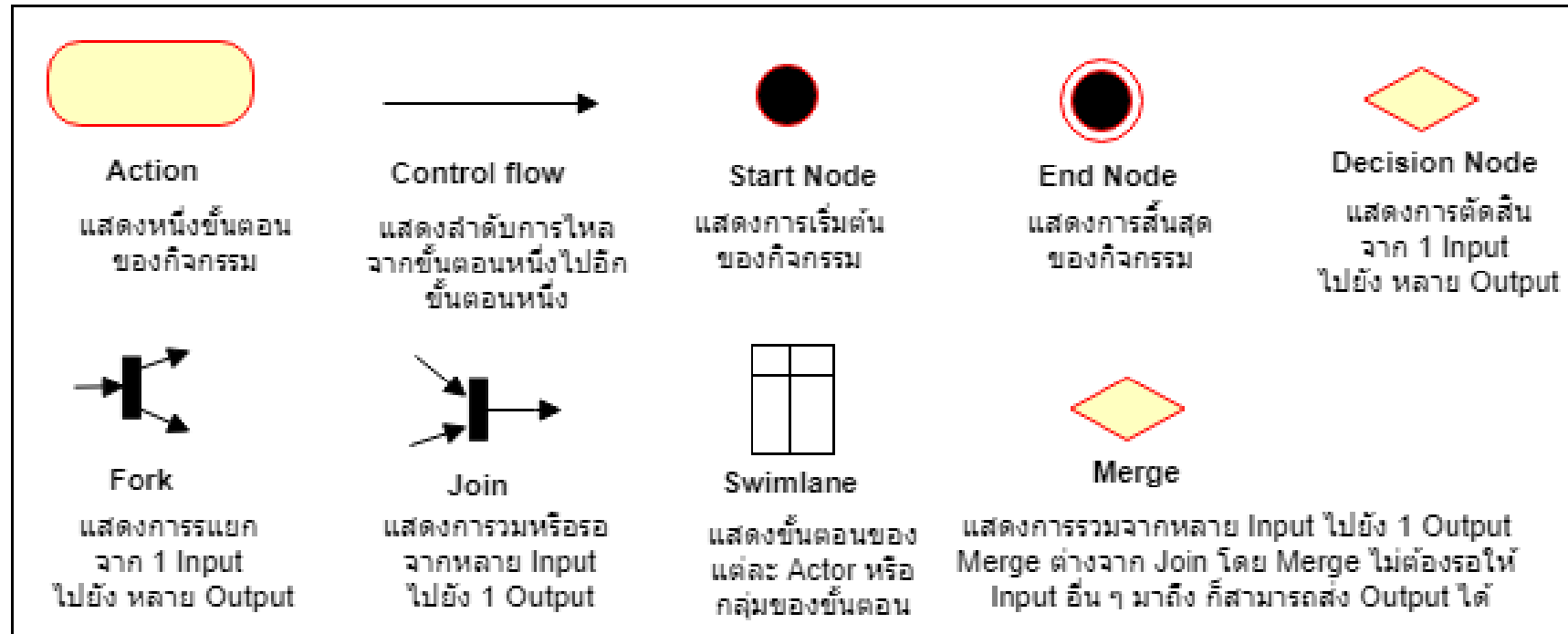
ภาพที่ 3-16 : ความสัมพันธ์แบบ Aggregation



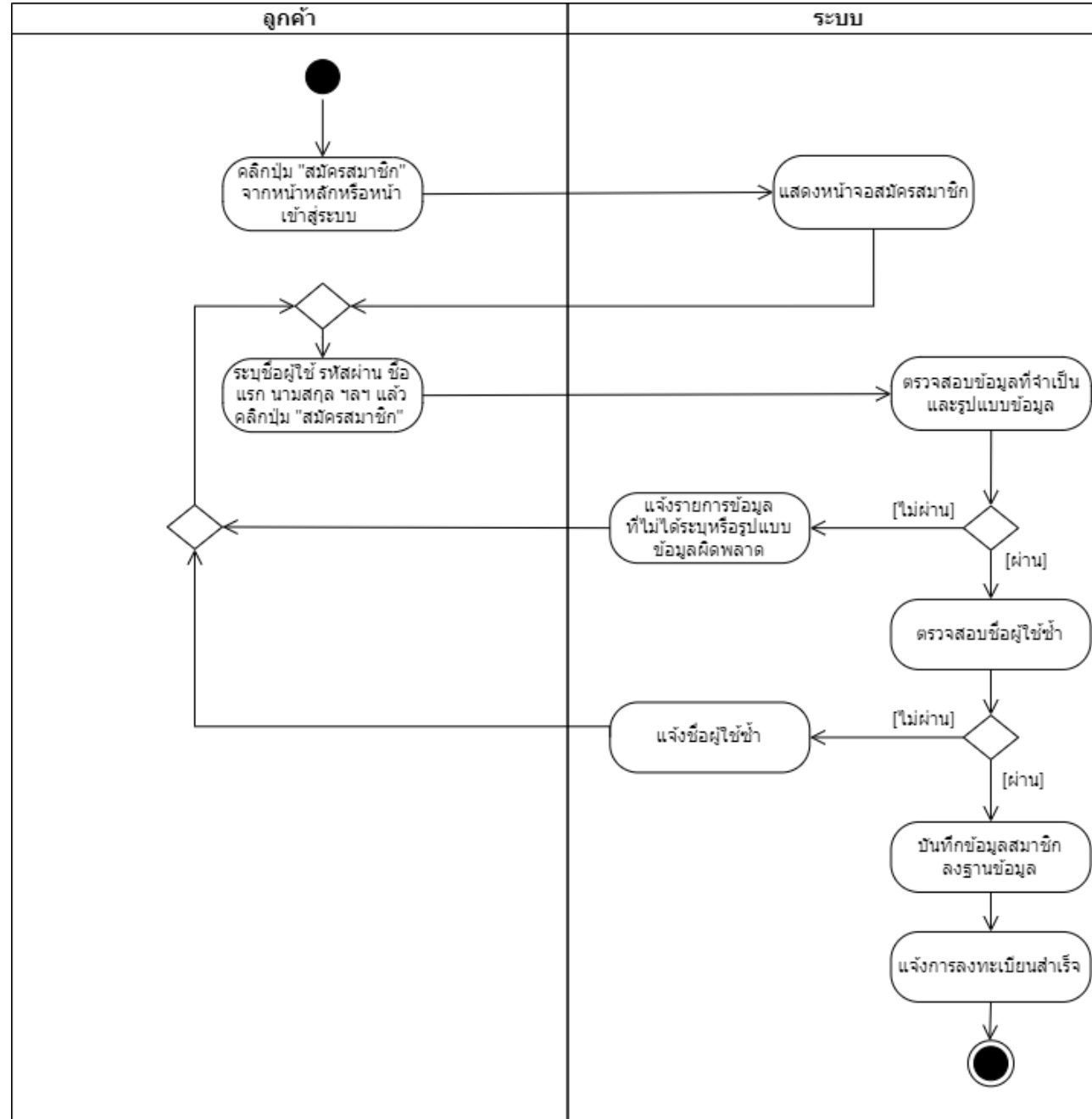
ภาพที่ 3-17 : Class Diagram ของระบบซื้อขายออนไลน์

3.5 Activity Diagram

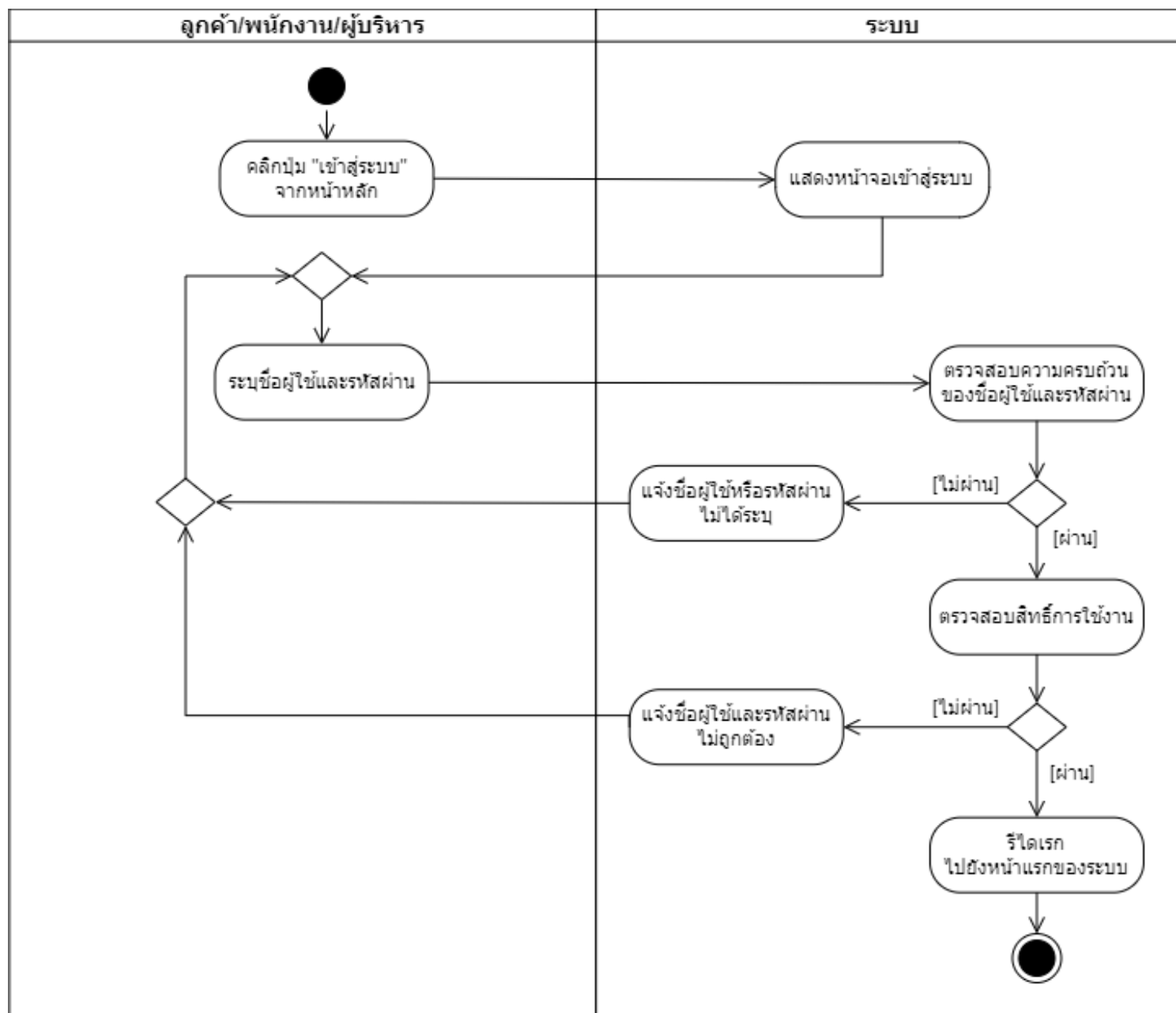
แอกทิวิตีไดอะแกรม (Activity Diagram) เป็นแผนภาพที่ใช้แสดงลำดับเหตุการณ์ของกิจกรรมในระบบ จากขั้นตอนหนึ่งไปยังอีกขั้นตอนหนึ่ง และแสดงให้เห็นว่าแต่ละขั้นตอนใคร (อาจเป็นคนหรือระบบ) เป็นผู้ดำเนินการ



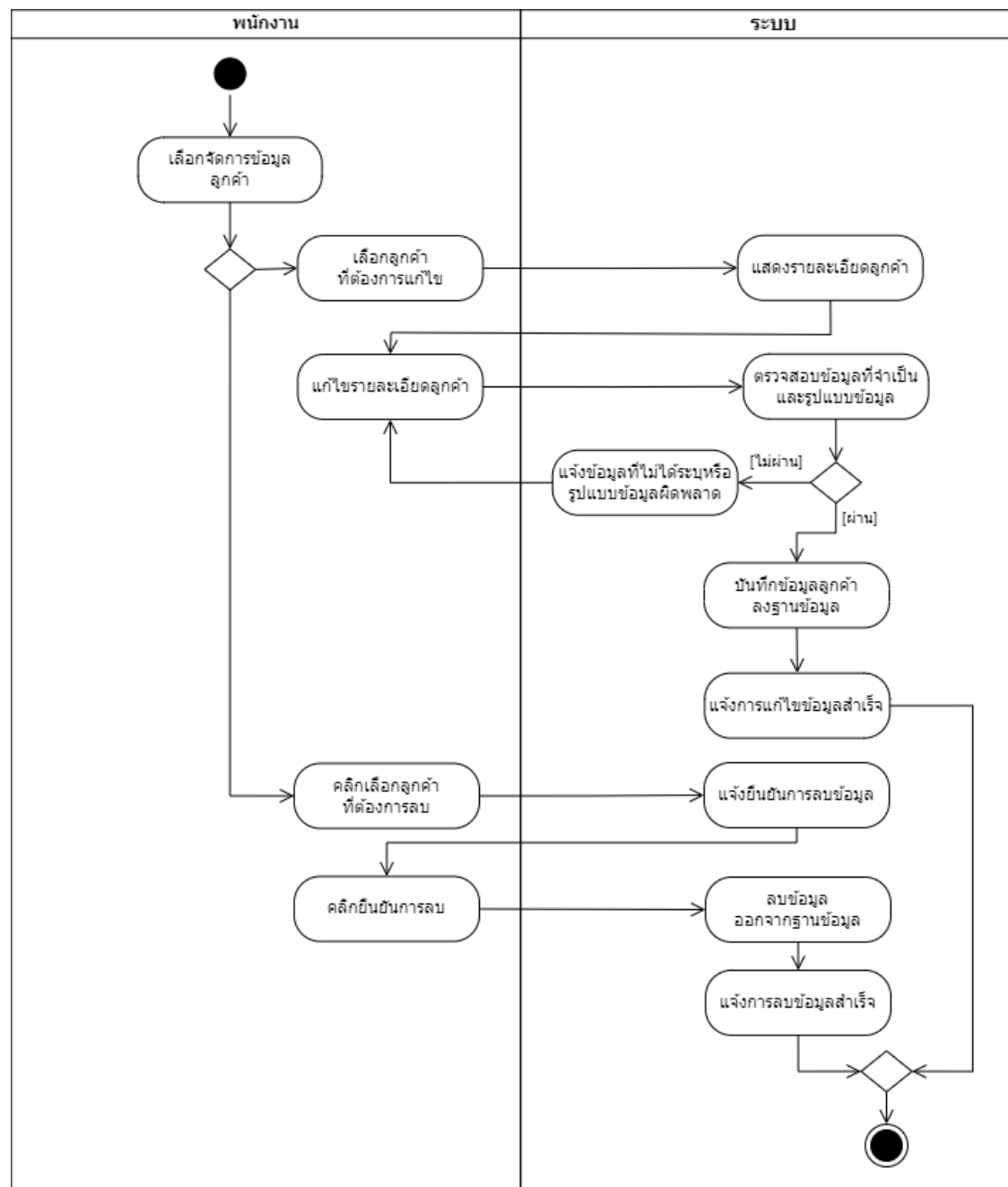
ภาพที่ 3-18 : สัญลักษณ์ของ Activity Diagram



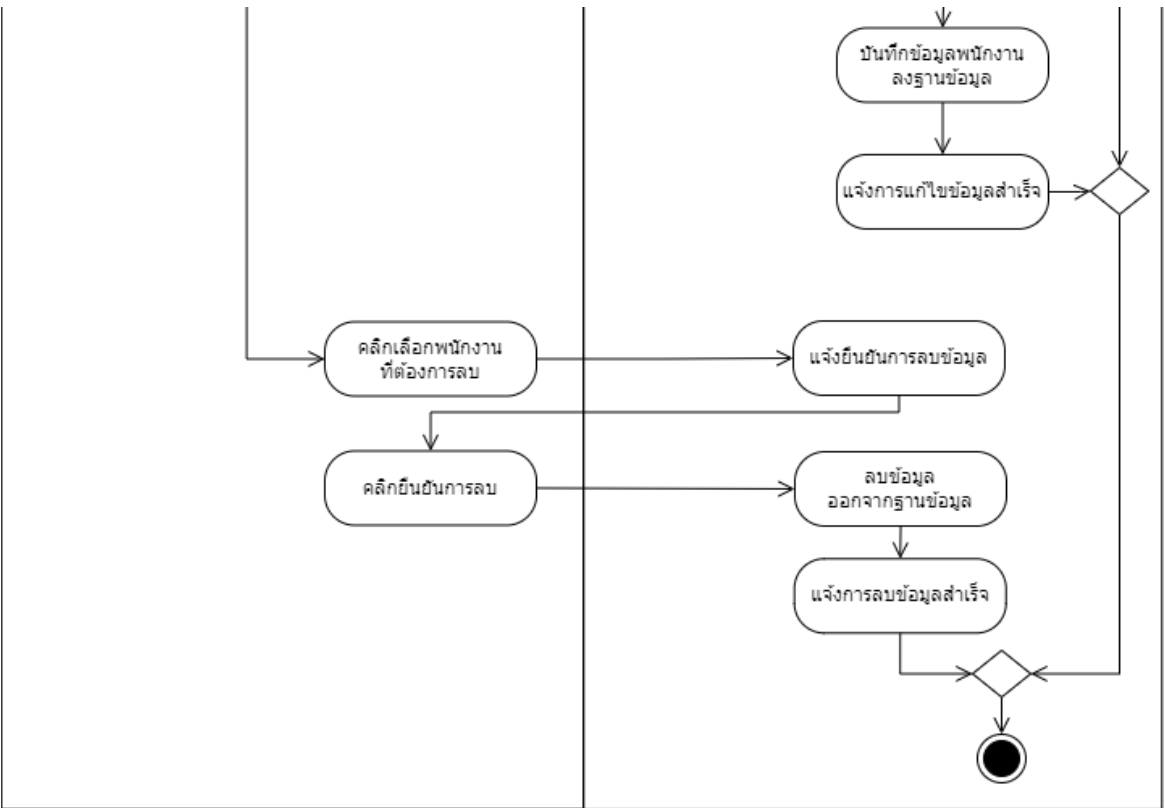
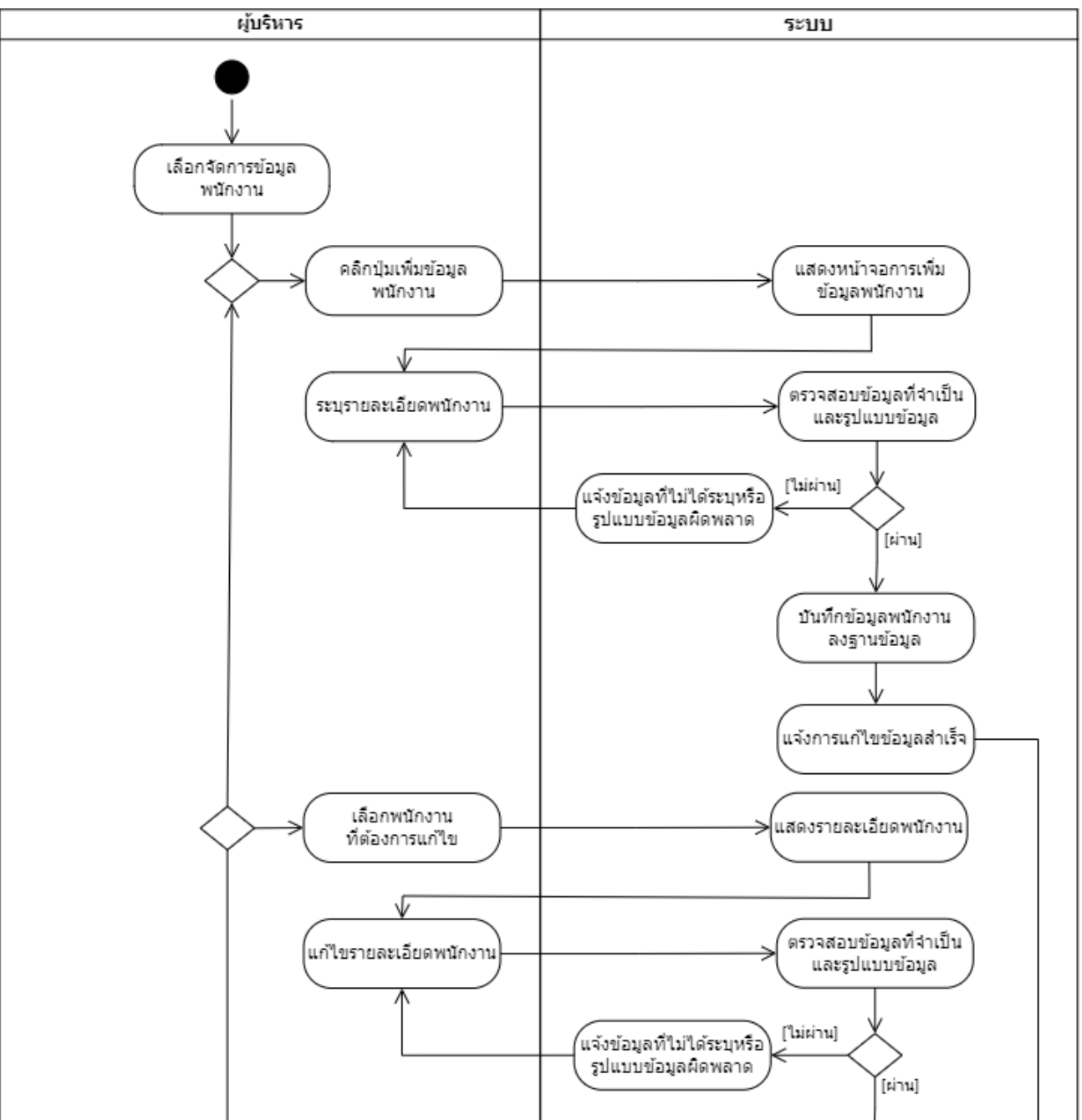
ภาพที่ 3-19 : Activity Diagram ของการลงทะเบียน



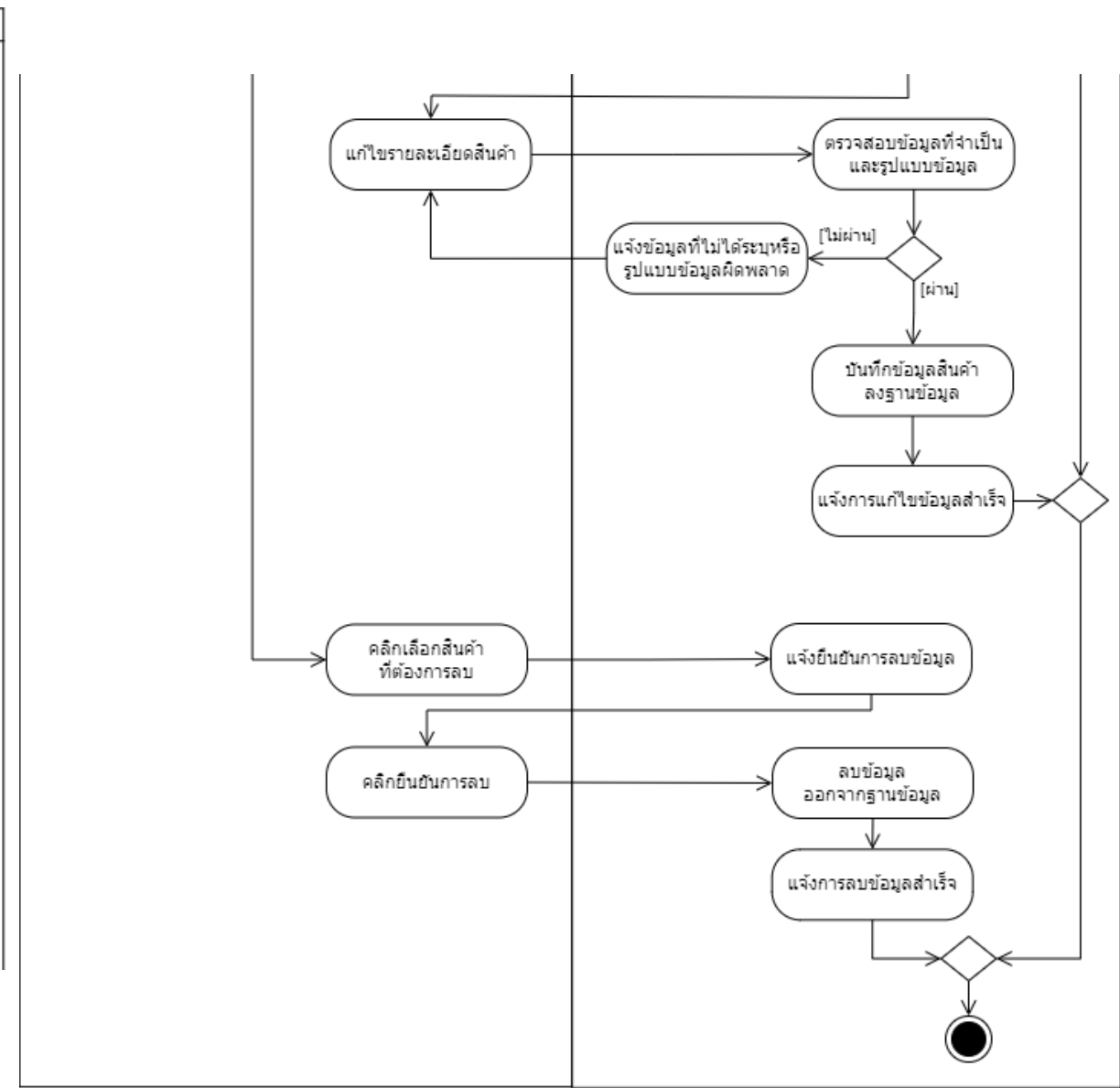
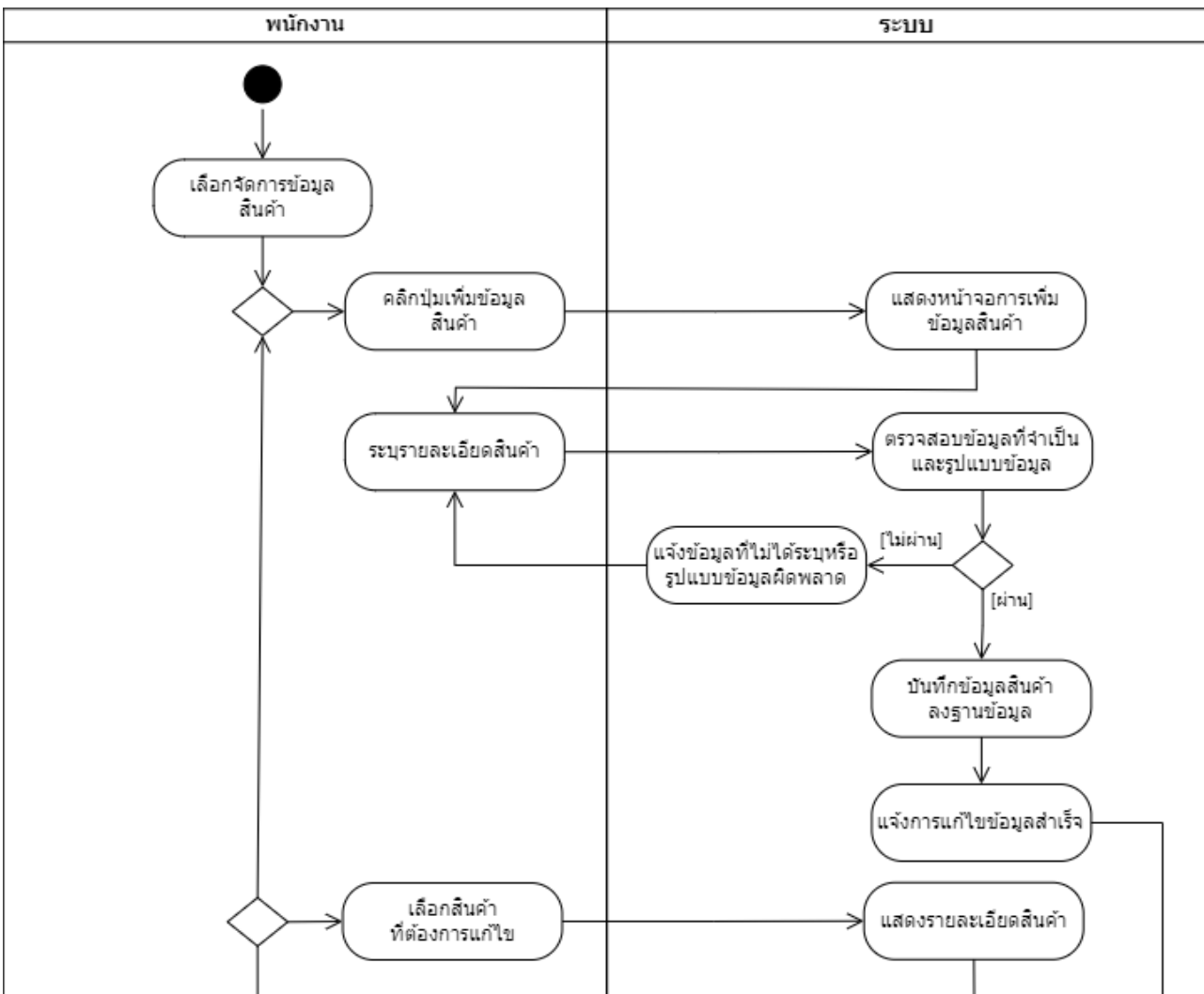
ภาพที่ 3-20 : Activity Diagram ของการเข้าสู่ระบบ



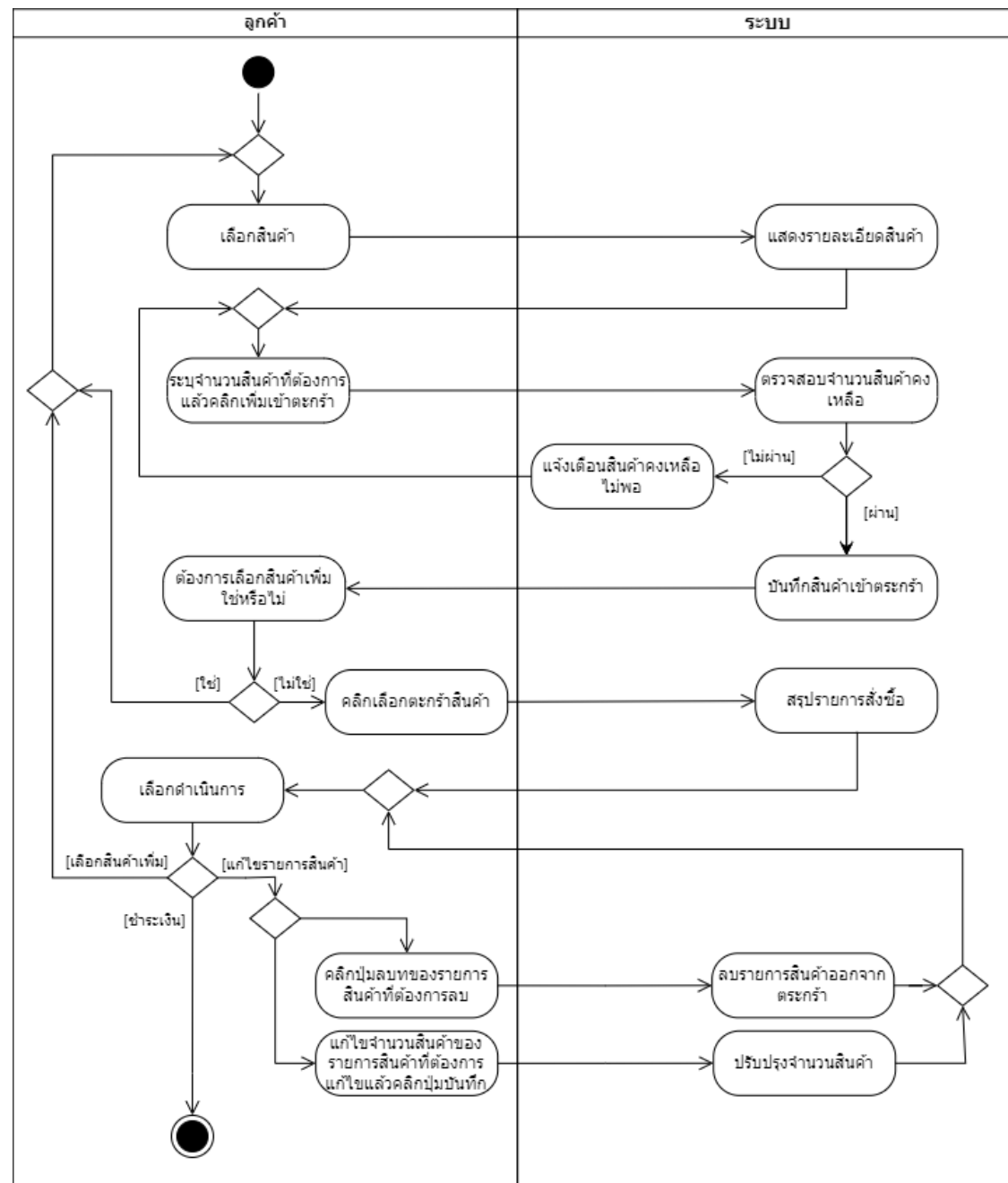
ภาพที่ 3-21 : Activity Diagram ของการจัดการข้อมูลลูกค้า



ภาพที่ 3-22 : Activity Diagram ของการจัดการข้อมูลพนักงาน



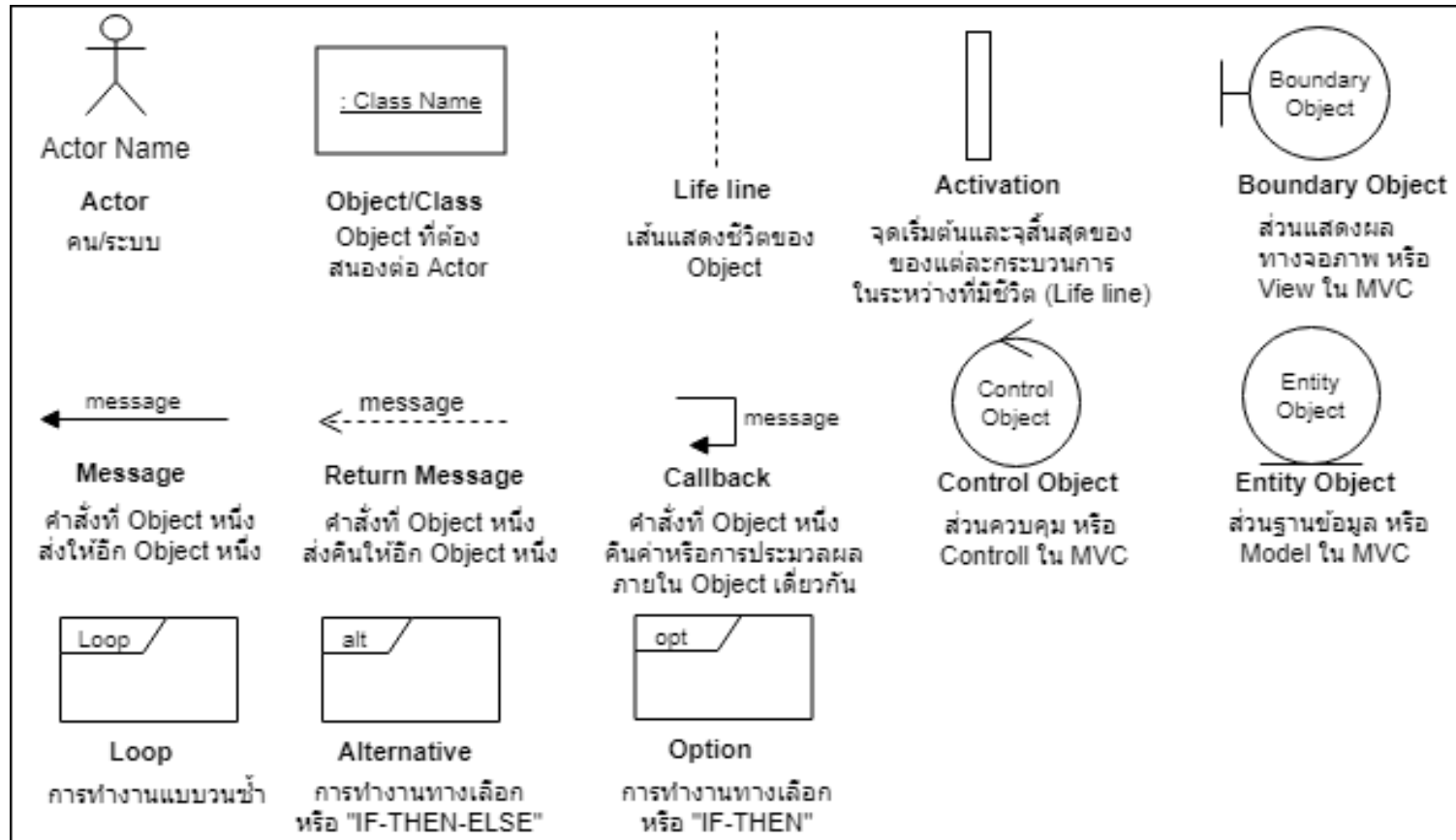
ภาพที่ 3-23 : Activity Diagram ของการจัดการสินค้า



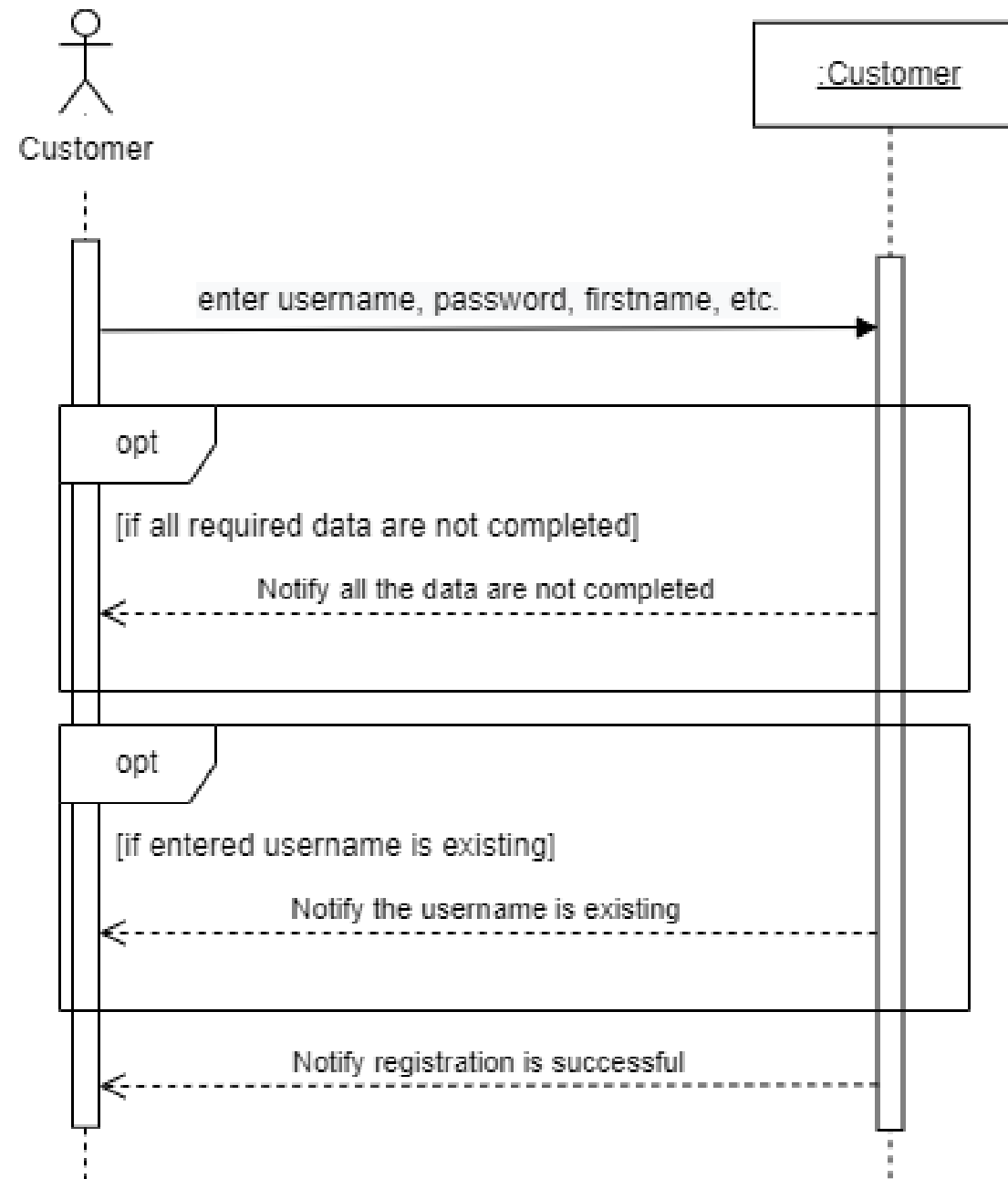
ภาพที่ 3-24 : Activity Diagram ของการสั่งซื้อ

3.6 Sequence Diagram

ซีควเอนไดอะแกรม (Sequence Diagram) เป็นแผนภาพแสดงการทำงานร่วมกันระหว่าง Object เพื่อให้กระบวนการสำเร็จ โดยเฉพาะการส่ง Message ระหว่าง Object ตามลำดับเวลา (Sequence)



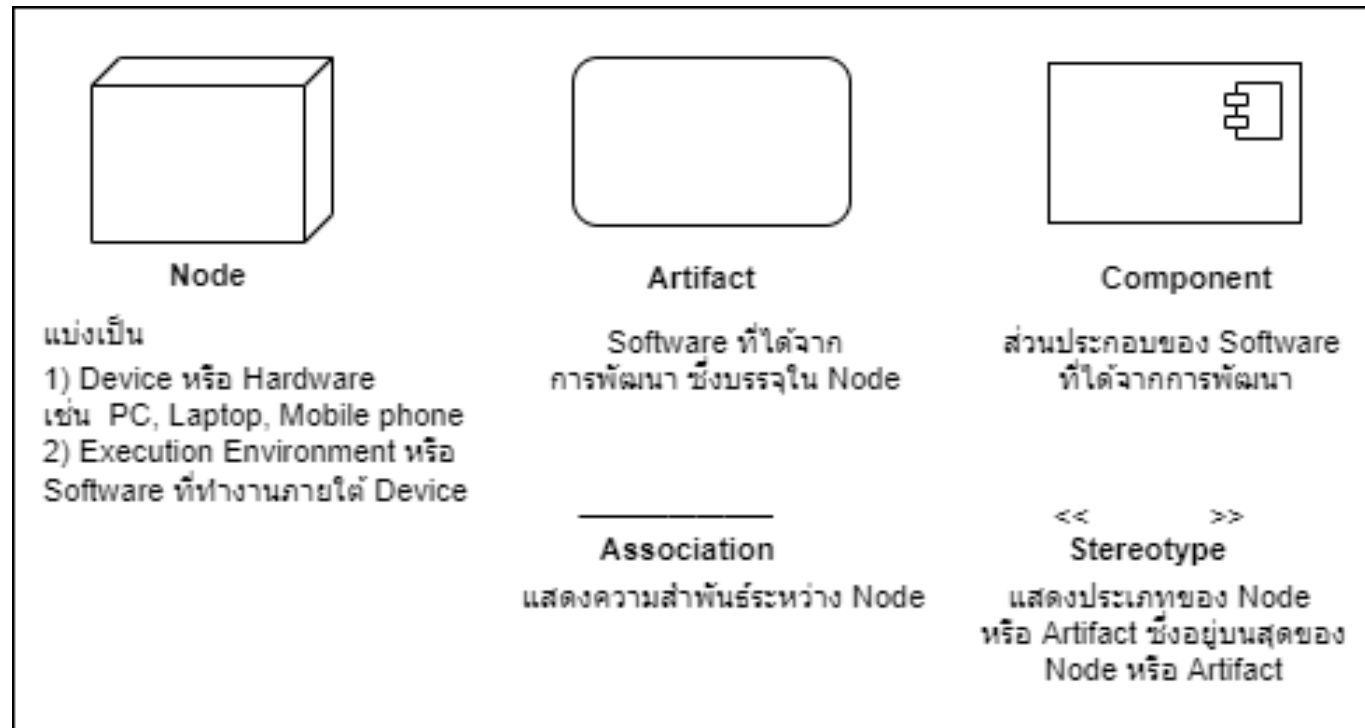
ภาพที่ 3-26: สัญลักษณ์ของ Sequence Diagram



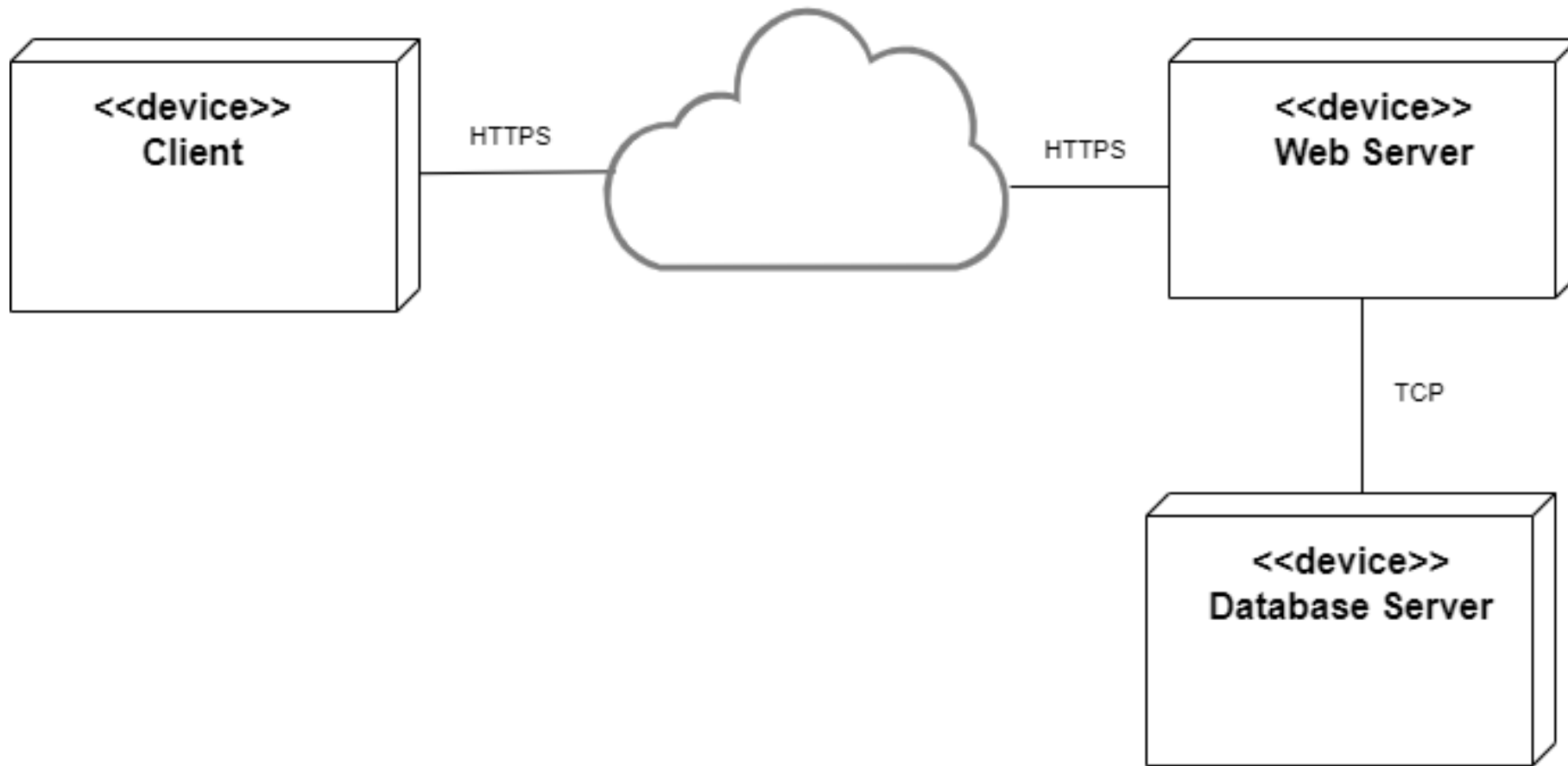
ภาพที่ 27 : Activity Diagram ของการลงทะเบียน

3.7 Deployment Diagram

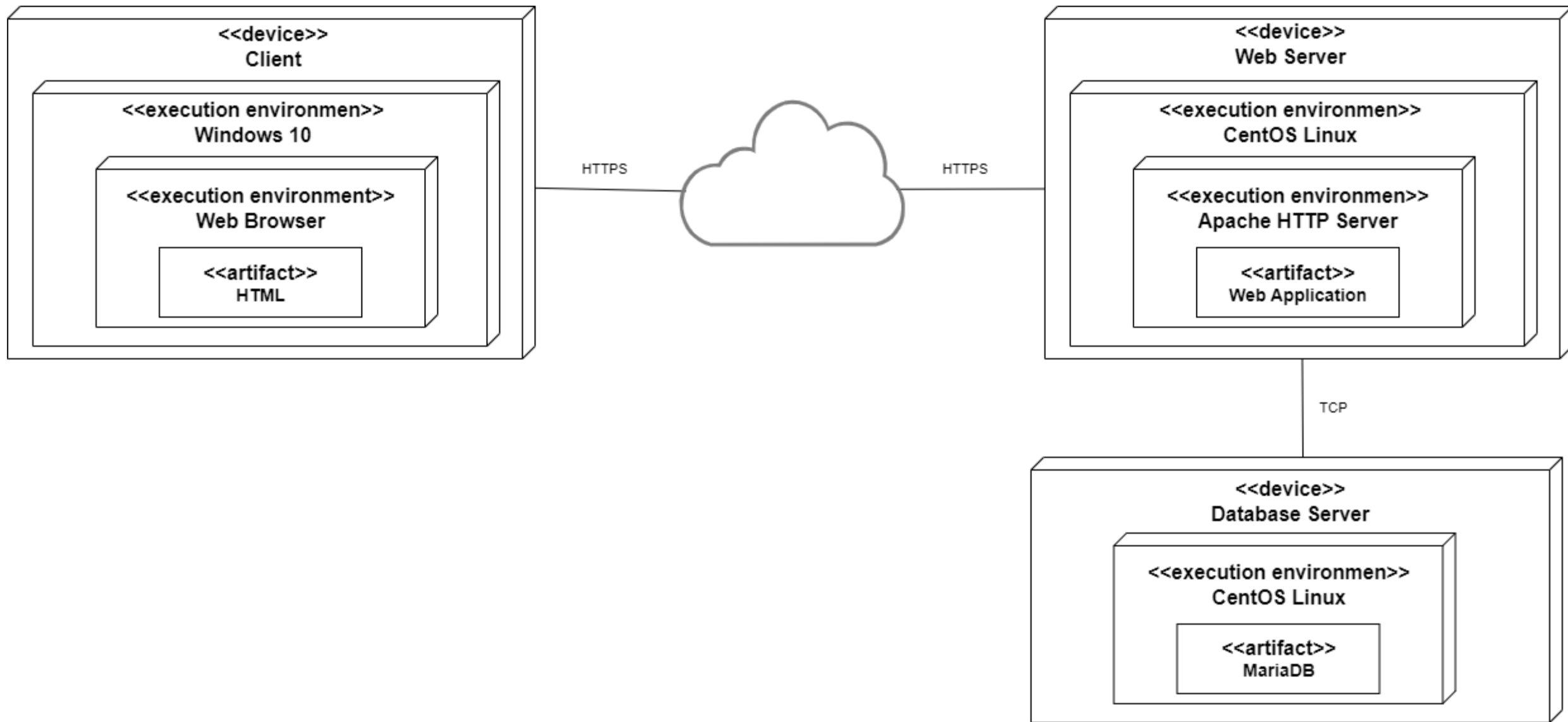
ดีพลอยเมนต์ไดอะแกรม (Deployment Diagram) เป็นแผนภาพแสดงสถาปัตยกรรมของระบบ ประกอบไปด้วย ฮาร์ดแวร์และซอฟต์แวร์ที่เกี่ยวข้องกับซอฟต์แวร์ที่พัฒนา โดยมองฮาร์ดแวร์และซอฟต์แวร์เหล่านั้นเป็น Object เช่นเดียวกันกับซอฟต์แวร์ที่พัฒนา



ภาพที่ 3-28 : สัญลักษณ์ของ Deployment Diagram



ภาพที่ 3-29 : สถาปัตยกรรมระบบแบบ Client-Server (แบบที่ 1)



ภาพที่ 3-30 : สถาปัตยกรรมระบบแบบ Client-Server (แบบที่ 2)

Do you have any questions?

